

1. INTRODUCTION ABOUT THE PROJECT

In today's fast-paced world of competitive sports, efficient tournament administration and athlete management have become critical challenges for organizations ranging from local clubs to state-level associations. The traditional methods of managing sports events—often relying on fragmented spreadsheets, paper-based scorecards, and manual scheduling—are increasingly prone to errors and inefficiencies. Despite the popularity of various sports, organizers often struggle to find a centralized, user-friendly platform that can handle the distinct rules and scoring complexities of different disciplines, from team sports like Cricket and Football to individual categories like Weightlifting. This is where the **Sports Management System** comes in.

This project bridges the gap between chaotic event coordination and professional sports administration by offering a comprehensive, feature-rich dashboard for tournament managers and public users. Designed with a focus on data integrity, real-time updates, and usability, the platform provides a streamlined approach to organizing sporting events. Managers can create tournaments, configure sport-specific rules (such as weight classes or batting roles), schedule fixtures across various venues, and record complex match results. Unlike static scheduling tools, this system integrates dynamic scoring logic—automatically calculating totals for Weightlifting or tracking wickets for Cricket—to ensure accuracy.

The goal of this project is to create a unified solution that empowers sports organizers to take control of their events with digital precision. By leveraging robust backend logic and dynamic database schemas, the platform not only provides a seamless roster management experience but also ensures that fans and public users have instant access to match schedules, team details, and live status updates.

As the necessity for digital transformation in the sports sector continues to rise, this system is built to meet these growing needs. Its responsive design, intuitive manager dashboard, and adaptable configuration capabilities make it a powerful tool for users ranging from college sports departments to independent tournament directors.

From a technical perspective, the platform's development is rooted in modern web development principles, utilizing **HTML5**, **Jinja2 templates**, **Bootstrap**, and **jQuery** to create a responsive and interactive frontend. The application is powered by a robust backend built with **Python 3.x** and the **Flask framework**, employing the Application Factory pattern to ensure scalability and modularity. The data layer is managed using **MySQL with SQLAlchemy ORM**, utilizing advanced JSON columns to store flexible schema definitions for different sports. The project follows a modular development approach, utilizing Flask Blueprints to organize routing and logic for Administrators, Managers, and Public Users, ensuring maintainability and ease of future enhancements.

By combining ease of use, technical flexibility, and administrative depth, the Sports Management System seeks to stand out in the domain of sports technology. Whether an organizer is looking to host a local cricket cup or a state-level weightlifting championship, this platform offers the tools and resources needed to succeed in today's digital sports environment.

2.OBJECTIVE AND SCOPE OF PROJECT

The **Sports Management System** has the following objectives:

- **To provide** a comprehensive mechanism for managing multiple types of sports (Team and Individual) within a single platform, utilizing dynamic configuration schemas to handle unique rules for Cricket, Football, Weightlifting, and more.
- **To offer** a centralized dashboard for tournament managers to create events, select venues (e.g., specific stadiums in cities like Raipur or Bilaspur), and manage the entire lifecycle of a tournament from registration to completion.
- **To enable** managers to maintain accurate databases of teams and players, including capturing sport-specific details such as "Batting Style" for cricketers or "Weight Category" for weightlifters.
- **To empower** organizers with automated scoring and scheduling tools, ensuring that fixtures are generated correctly and match results (scores, goals, or lifts) are calculated and stored without manual error.
- **To provide** a secure and user-friendly platform with Role-Based Access Control (RBAC), ensuring that Managers have full administrative rights over their events while Public Users can view schedules and save their favorite tournaments.
- **To enhance** the spectator experience by offering a public-facing portal where users can browse upcoming fixtures, view team squads, and track the status of live or completed events.

The scope for developing the **Sports Management System** includes:

- **Managing** user authentication and session security, including secure registration, login, and distinct access levels for Administrators, Managers, and Public Users.
- **Developing** a responsive, mobile-friendly interface using Bootstrap that allows managers to update scores and manage schedules from the field using tablets or smartphones.
- **Implementing** a flexible database architecture using MySQL and JSON fields to store variable data structures for different sports without needing separate database tables for every new sport added.
- **Integrating** client-side logic (AJAX/JavaScript) for dynamic modals and forms, allowing for real-time score entry (e.g., recording Snatch and Clean & Jerk attempts) without refreshing the page.
- **Creating** a robust backend architecture using Python (Flask) to handle data persistence, complex relationships between Events, Teams, and Players, and the efficient retrieval of fixture data.

3(A). INTRODUCTION OF HTML AND CSS

HTML (Hypertext Mark-up Language) and CSS (Cascading Style Sheets) are essential technologies that form the foundation of web development, enabling the creation of visually engaging and well-structured web pages. Much like Visual Basic in application development, HTML and CSS are integral in defining the structure and presentation of web content.

HTML (Hypertext Mark-up Language):

HTML serves as the fundamental framework of web pages, providing a structured mark-up language to define content and layout. Developers use HTML to create documents by utilizing tags that categorize and organize different elements on a webpage, such as headings, paragraphs, images, links, forms, and more.

CSS (Cascading Style Sheets):

CSS complements HTML by allowing developers to manage the visual styling of web pages. It introduces style rules that dictate how HTML elements should be displayed across various devices. By separating content from design, CSS streamlines the creation

of consistent and visually appealing user interfaces.

Together, HTML and CSS provide the essential building blocks for crafting an engaging and user-friendly web experience. While HTML defines the structure, CSS enhances the design, offering a flexible and responsive

layout. This synergy supports the principles of Rapid Application Development (RAD), simplifying the process of designing and styling web interfaces efficiently.

3(B). INTRODUCTION OF JAVASCRIPT

JavaScript is a versatile and powerful programming language primarily employed in web development. It is a key component of modern web browsers, empowering developers to create dynamic and interactive user interfaces. Often abbreviated as JS, JavaScript plays a pivotal role in enhancing web page functionality through client-side scripting.

Key Features of JavaScript:

Client-Side Scripting:

JavaScript is predominantly used for client-side scripting, meaning it executes directly within the user's web browser. This capability enables developers to craft dynamic content, validate forms, handle events, and manipulate the Document Object Model (DOM), allowing webpage content to be updated dynamically without the need for a page reload.

Object-Oriented:

JavaScript is an object-oriented language, supporting core concepts such as encapsulation, inheritance, and polymorphism. These features allow developers to organize and modularize their code effectively, facilitating better code management and scalability.

Interactivity and Dynamic Content:

JavaScript enhances web pages by enabling real-time responsiveness to user interactions. Developers can implement features like sliders, pop-ups, and form validations, while also facilitating dynamic content loading and updating, contributing to more engaging and user-friendly websites.

Event-Driven Programming:

JavaScript follows an event-driven programming model, wherein developers define functions that execute in response to specific events such as button clicks, mouseovers, or form submissions. This paradigm improves user experience by making web pages more interactive and responsive to user actions.

Cross-Browser Compatibility:

JavaScript is supported by all major web browsers, including Chrome, Firefox, Safari, and Edge, ensuring consistent functionality across diverse platforms and devices.

JavaScript is an indispensable technology in modern web development, frequently working alongside HTML and CSS to build interactive web applications. Over time, JavaScript has expanded beyond its initial use in web browsers, now also playing a significant role in server-side development (via Node.js), mobile app development, and other application domains.

3(C). PYTHON AND FLASK FRAMEWORK

Python is a high-level, interpreted, general-purpose programming language known for its emphasis on code readability and simplicity. Created by Guido van Rossum and first released in 1991, Python has grown to become one of the most popular programming languages in the world, powering everything from simple scripts to complex machine learning algorithms.

For this project, we utilize Flask, a micro web framework written in Python. Created by Armin Ronacher in 2010, Flask is classified as a "microframework" because it does not require particular tools or libraries to function. It has no default database abstraction layer, form validation, or other components where pre-existing third-party libraries provide common functions. However, Flask supports extensions that can add application features as if they were implemented in Flask itself.

Python code in a web context is executed via a WSGI (Web Server Gateway Interface) server. In our architecture, the Flask application factory (`create_app`) initializes the application, managing routes via Blueprints and handling database interactions through SQLAlchemy. Unlike older CGI scripts, this modern stack ensures that the application remains persistent and efficient in handling concurrent user requests.

While Python is often associated with data science and automation, its web capabilities are robust. Major platforms like Netflix, Uber, and Pinterest utilize Python for its scalability and ease of integration. The standard Python interpreter (CPython) is open-source and managed by the Python Software

Foundation.

According to the TIOBE Index and GitHub Octoverse reports, Python consistently ranks as one of the top programming languages globally. Its ecosystem allows for the seamless integration of mathematical calculations (essential for our Compound Interest module) and data processing, making it the ideal choice for a financial dashboard.

KEY FEATURES

- **Microframework Architecture (Performance):** Flask is lightweight and modular. Unlike monolithic frameworks that force a specific directory structure and dependencies, Flask allows the developer to keep the core application simple and extensible. This results in faster startup times and a codebase that is easier to navigate and maintain.
- **Open Source:** Both Python and Flask are open-source technologies with permissive licenses (BSD/MIT). This means the entire stack—from the interpreter to the web framework and ORM—is free to use, modify, and distribute, significantly reducing the development cost of the project.
- **Readable Syntax:** Python is often described as "executable pseudocode." Its syntax is clear, concise, and English-like, which reduces the cognitive load on developers. This readability ensures that the complex logic behind expense tracking and interest calculations remains understandable and maintainable.
- **Jinja2 Templating (Embedded Logic):** Flask utilizes the Jinja2

templating engine, which allows Python-like logic to be embedded within HTML files. This enables the server to render dynamic content—such as user-specific dashboards and expense tables—securely and efficiently before sending the final HTML to the client's browser.

- **Platform Independence:** Python is cross-platform by design. The Personal Finance Management System can be developed on Windows and deployed on Linux (e.g., Ubuntu with Nginx/Gunicorn) without requiring code changes. This portability ensures flexibility in hosting and deployment options.
- **Database Support (SQLAlchemy):** This project uses Flask-SQLAlchemy, an Object Relational Mapper (ORM) that provides a high-level abstraction for database interactions. It supports a wide range of databases, including the MySQL database used in our production environment. The ORM allows us to interact with the database using Python classes and objects rather than raw SQL queries, improving security and maintainability.
- **Robust Error Reporting:** Flask comes with a built-in debugger and logger. During development, the Werkzeug debugger provides interactive tracebacks in the browser if a crash occurs, allowing for rapid identification and resolution of bugs within the expense logic or routing.
- **Dynamic Typing:** Python is dynamically typed, meaning variable types

are determined at runtime. This allows for rapid prototyping and flexibility when handling various data inputs, such as different currency formats or user profile details, without the strict boilerplate code required by statically typed languages.

- Security: Security is a core component of the Flask ecosystem. Our project leverages:
- Jinja2 Auto-escaping: To automatically protect against Cross-Site Scripting (XSS) attacks by escaping untrusted input in templates.
- Extensibility (Control): Flask gives developers granular control over the application. Features are added only when needed. For instance, we explicitly added Flask-Migrate for database schema management and Flask-Login for authentication. This "opt-in" approach prevents bloat and ensures the application remains optimized for its specific financial tracking purpose.
- Thriving Community: The Python and Flask communities are vast and active. Extensive documentation, third-party extensions (like Flask-WTF for forms), and community support ensure that any technical challenge encountered during the development of the Personal Finance Management System can be resolved quickly.

3(D). INTRODUCTION OF MYSQL

What is a Database?

A database is a specialized application designed to store, manage, and organize data in a structured manner. It provides distinct APIs that allow users to create, access, manage, search, and replicate the data it contains. While other types of data storage systems, such as file systems or large hash tables in memory, can be used, they generally do not offer the same speed and ease of data retrieval and manipulation as a dedicated database system.

In modern applications, relational database management systems (RDBMS) are commonly used to handle large volumes of data. The term "relational" refers to the way data is organized into tables, where relationships between the data are established using primary keys and foreign keys.

Relational Database Management System (RDBMS)

An RDBMS is software that facilitates the implementation and management of relational databases. It enables the creation of databases with tables, columns, and indexes while ensuring the following functions:

- **Table Structure:** Allows the creation and organization of tables to store data.
- **Referential Integrity:** Maintains the integrity of relationships

between tables using primary and foreign keys.

- **Index Management:** Automatically updates indexes to optimize data retrieval.
- **SQL Query Processing:** Interprets SQL queries and retrieves data by combining information from various tables.

RDBMS Terminology

Before exploring specific database systems like MySQL, it is important to understand some key terminology used in RDBMS:

- **Database:** A collection of related tables that store data.
- **Table:** A table is a collection of rows and columns, representing data in a structured format similar to a spreadsheet.
- **Column:** A column contains data of a specific type or category, such as customer names, addresses, or product prices.
- **Row:** A row (also known as a tuple, entry, or record) is a set of related data within a table, often representing an individual record.
- **Redundancy:** The practice of storing data multiple times to improve system performance. While redundancy can speed up access, it may introduce challenges in data consistency.
- **Primary Key:** A primary key is a unique identifier for each record in a table. It ensures that each row in a table is distinguishable from others, and no two rows can have the same primary key value.
- **Foreign Key:** A foreign key is used to create a relationship between two

tables. It refers to the primary key of another table, ensuring referential integrity between the two tables.

- **Compound Key (Composite Key):** A compound key is a primary key that consists of two or more columns, used when a single column cannot uniquely identify records in a table.
- **Index:** An index in a database is a data structure that allows for faster retrieval of data, similar to an index at the back of a book. It speeds up query execution by reducing the amount of data that needs to be scanned.
- **Referential Integrity:** Referential integrity ensures that foreign key values in a table always point to valid, existing rows in another table, preventing orphaned records and maintaining data consistency.

What is SQL?

SQL (Structured Query Language) is the standard language used for managing and manipulating data in a relational database management system (RDBMS). It provides an interface for accessing, updating, deleting, and managing database structures and data. SQL is the foundation of interacting with databases, enabling users to perform a wide range of operations on the data they store.

SQL is divided into four main subsets:

1. **DDL (Data Definition Language):** DDL is used to define and manage the structure

of a database. It allows the creation, alteration, and deletion of database objects such as tables, views, and schemas. Key commands include:

- **CREATE:** To create new database objects.
- **ALTER:** To modify existing database objects.
- **DROP:** To delete database objects.

2. **DML (Data Manipulation Language):** DML deals with the manipulation of data within the database. It allows for adding, updating, deleting, and querying data. Common DML commands include:

- **SELECT:** To retrieve data from one or more tables.
- **INSERT:** To add new data into a table.
- **UPDATE:** To modify existing data.
- **DELETE:** To remove data from a table.

3. **DCL (Data Control Language):** DCL is used to control access to data within the database. It helps manage user permissions, allowing for secure access and data management. Main DCL commands include:

- **GRANT:** To give a user specific access rights to database objects.
 - **REVOKE:** To remove previously granted permissions.
4. **TCL (Transaction Control Language):** TCL is used to manage transactions in a database. A transaction is a sequence of operations performed as a single unit. TCL ensures that transactions are handled properly, preserving data integrity. Key TCL commands include:
- **COMMIT:** To save all changes made during the transaction.
 - **ROLLBACK:** To undo changes made during the transaction if an error occurs.
 - **SAVEPOINT:** To set a point within a transaction to which you can roll back.
 - **SET TRANSACTION:** To configure transaction properties

MySQL

MySQL is a widely used relational database management system (RDBMS) developed and maintained by MySQL AB, a Swedish company. It is known for being fast, reliable, and easy to use, making it a popular choice for both small businesses and large enterprises. MySQL has several advantages that contribute to its growing popularity:

- **Open Source:** MySQL is released under an open-source license, meaning it is free to use and modify.
- **Powerful:** Despite being free, MySQL provides a range of features that rival some of the most expensive commercial database management systems. It can handle substantial databases and a variety of data processing tasks.
- **Standardized SQL Support:** MySQL uses a standard form of SQL, making it compatible with most database applications.
- **Cross-Platform Compatibility:** MySQL can run on various operating systems, including Windows, Linux, and macOS, and supports multiple programming languages like PHP, PERL, C, C++, and Java.
- **Speed:** MySQL is known for its quick performance, especially with large datasets. It is optimized for high-speed operations.
- **Scalability:** MySQL can handle large databases with ease. It supports up to 50 million rows per table, with the theoretical file size limit reaching 8 million terabytes.
- **Customization:** Being open source, MySQL can be customized and extended to fit specific needs. Developers can modify the software to suit their particular environment or requirements.

4. DEFINITION OF PROBLEM

The Sports Management System faces unique challenges related to the diversity of sporting rules, real-time data accuracy, and user hierarchy. Unlike a standard data entry application, a sports platform must handle widely varying logic—from the complex over-by-over tracking in Cricket to the specific weight-class attempts in Weightlifting—within a single unified database. Below are the primary problems this project addresses:

1. Handling Diverse Sporting Rules:

- **Challenge:** Integrating multiple sports (Cricket, Football, Kabaddi, Weightlifting) that have completely different scoring metrics (Runs/Wickets vs. Goals vs. Kilograms lifted).
- **Impact:** A rigid database structure would fail to accommodate these differences, requiring a dynamic schema approach to ensure the system is flexible.

2. Data Integrity and Validation:

- **Challenge:** Minimizing human error during match scoring. For example, ensuring a manager cannot enter a "7th ball" in a standard cricket over or enter a score for a player not listed in the squad.
- **Impact:** Incorrect data entry leads to inaccurate leaderboards and match results, diminishing the credibility of the tournament.

3. Role-Based Access Control (RBAC):

- **Challenge:** Strictly enforcing security boundaries where **Managers** have full control over their specific events (creating fixtures, editing scores), while **Public Users** are restricted to read-only access.
- **Impact:** Without robust session management, unauthorized users could tamper with match schedules or results.

4. Real-Time Scoring and Visualization:

- **Challenge:** Rendering accurate, up-to-date scorecards for public viewers. A Weightlifting scoreboard needs to show "Snatch" and "Clean & Jerk" columns, while a Cricket scoreboard requires a completely different layout.
- **Impact:** The interface must dynamically adapt based on the sport type to provide a meaningful viewing experience.

5. Scheduling and Venue Management:

- **Challenge:** preventing scheduling conflicts where two matches might be booked for the same venue (e.g., "Raipur Stadium") at the same time.
- **Impact:** Efficient conflict detection is required to ensure smooth tournament logistics.

6. Automated Calculation Precision:

- **Challenge:** Removing the need for manual math. In Weightlifting, the "Total" (Best Snatch + Best C&J) must be auto-calculated. In Cricket, run rates and total overs must be computed automatically.

- **Impact:** Manual calculations are prone to error; the system must handle the logic to ensure instant and precise results.

7. Mobile Compatibility for Field Use:

- **Challenge:** Ensuring the "Manager Dashboard" is fully responsive so tournament organizers can update scores directly from the field using smartphones.
- **Impact:** If the scoring interface is not mobile-friendly, managers are forced to use paper sheets and digitize data later, delaying public updates.

8. Database Scalability:

- **Challenge:** Ensuring the database performs efficiently as the number of players, teams, and historical match records grows over multiple seasons.
- **Impact:** Optimizing relationships between `Events`, `Fixtures`, and `Scores` (using JSON storage) to prevent slow load times.

In sum, the Sports Management System addresses the complexity of modern tournament administration by providing a flexible, secure, and automated platform that standardizes how different sports are managed and viewed.

5. SYSTEM ANALYSIS

System analysis for the Sports Management System involves understanding the requirements of tournament organizers and sports enthusiasts. The system must bridge the gap between complex administrative tasks and a simple, engaging user experience. The analysis phase involved studying the specific rules of target sports (Cricket, Weightlifting, etc.) and identifying the technical architecture required to support them dynamically.

Here is the System Analysis for the Sports Management System:

1. Problem Identification:

- **Fragmented Management:** Organizers often use disconnected tools (paper, Excel, WhatsApp) to manage tournaments, leading to data loss and miscommunication.
- **Lack of Standardization:** There is no single platform that allows a college or club to manage a Cricket tournament and a Weightlifting championship simultaneously.
- **Delayed Information:** Spectators often have to wait until the end of the day to see results, lacking a real-time digital connection to the event.

2. User Requirements:

- **Tournament Managers (Admin/Organizers):** Need a powerful dashboard to register teams, generate fixtures, and enter scores quickly without technical knowledge.
- **Public Users (Spectators/Players):** Require instant access to match schedules, team squads, and live score updates without needing to log in.

- **System Administrators:** Need oversight of all registered managers and the ability to maintain the master list of supported sports and venues.

3. System Requirements:

- **Functional:**
 - **Dynamic Sport Configuration:** The ability to define rules (e.g., "Weight Classes", "Overs") via JSON schemas.
 - **Fixture Generation:** Tools to pair teams and schedule matches at specific venues.
 - **Scoring Engine:** Dedicated logic for calculating winners based on sport-specific rules.
 - **Search & Filter:** Ability to find events by City (e.g., "Raipur") or Sport.
- **Non-Functional:**
 - **Flexibility:** The database must support adding new sports without rewriting the core code.
 - **Security:** Password hashing (Werkzeug) and session protection to secure user accounts.
 - **Performance:** AJAX-driven interfaces to ensure scoring updates don't require full page reloads.

4. System Architecture:

- **Framework:** The system follows the **Model-View-Controller (MVC)** pattern (implemented as MVT in Flask).

- **Frontend:** Built using **HTML5, Bootstrap 5, and jQuery**. It uses dynamic templates (Jinja2) to render different scoring forms based on the sport type.
- **Backend:** Powered by **Python (Flask)**. It utilizes **Blueprints** to separate public routes from manager administrative routes.
- **Database:** **MySQL** with **SQLAlchemy ORM**. A key architectural decision was using **JSON Columns** for `score_data` and `config_schema`, allowing the system to store unstructured data (like Cricket scorecards) alongside structured relational data (Users, Events).

5. Data Flow:

- **Input:** Managers input data via dynamic forms (e.g., entering "100kg" for a Snatch attempt).
- **Processing:** Flask routes validate this input against the sport's schema and calculate derived metrics (e.g., Total Score).
- **Storage:** Data is serialized and stored in MySQL.
- **Output:** The frontend fetches this data via API endpoints and renders it into human-readable scoreboards (Tables, Modals).

6. User Interface and Experience:

- **Dashboard-Centric:** Managers have a dedicated "Command Center" to see all their active tournaments.

- **Modal Interactions:** Scoring and Player Management are handled in pop-up modals to keep the user context intact without navigating away from the main fixture list.

7. Security Considerations:

- **Ownership Checks:** Every sensitive action (Delete Event, Update Score) includes a server-side check to ensure `current_user.id == event.manager_id`.
- **Data Sanitation:** Inputs are validated to prevent SQL injection and ensure data consistency (e.g., preventing duplicate player entries).

6.SYSTEM REQUIREMENTS

Minimum Hardware Requirements for Development

- **CPU:** Intel Core i3 (10th Gen) or AMD Ryzen 3 equivalent (Modern multi-core processors are recommended for running the Python interpreter and database services simultaneously).
- **Memory (RAM):** 8 GB (4 GB is the absolute minimum, but 8 GB is recommended to run Visual Studio Code, a MySQL server, and a web browser smoothly).
- **Hard Disk:** 256 GB SSD (Solid State Drive is highly recommended over HDD for faster project indexing and local server startup).
- **System Type:** 64-bit Operating System (Required for modern Python versions and efficient memory management).

Minimum Software Requirements for Development

- **Operating System:**
 - **Windows:** Windows 10 or 11 (64-bit).
 - **Linux:** Ubuntu 20.04 LTS or later (Preferred for deployment simulation).
 - **macOS:** macOS Catalina or later.
- **Database:** MySQL 8.0 Community Server (or MariaDB equivalent).
- **Web Server:**
 - **Development:** Werkzeug (Built-in Flask development server).
 - **Production Readiness:** Gunicorn (WSGI Server) behind Nginx.
- **Programming Language:** Python 3.10 or later.

- **Frontend Technologies:**
 - HTML5, CSS3 (Bootstrap 5 Framework).
 - JavaScript (ES6 Modules, Chart.js for visualization, FullCalendar.js).
- **IDE / Text Editor:** Visual Studio Code (with Python Extension) or PyCharm Community Edition.
- **Version Control:** Git (for tracking changes).
- **Browser:** Google Chrome or Mozilla Firefox (latest versions with Developer Tools).

Minimum Hardware and Software Requirements for Running the Website

On PCs (Desktops and Laptops)

- **CPU:** Intel Core i3 / AMD Ryzen 3 or equivalent (Sufficient for processing client-side JavaScript charts).
- **Memory (RAM):** 4 GB or higher (Required to handle the DOM manipulation for the dashboard and charts).
- **Hard Disk:** Not applicable (Client-side storage relies on browser cache/cookies; the application is hosted on the server).
- **Operating System:**
 - Windows: Windows 10 or later.
 - macOS: macOS Big Sur or later.
 - Linux: Any modern distribution with a GUI.

- **Web Browser:** Modern browsers supporting ES6 JavaScript and Canvas API (e.g., Google Chrome, Mozilla Firefox, Microsoft Edge, Safari). **Internet Explorer is not supported.**

On Mobile Devices (Phones and Tablets)

- **CPU:** Quad-core processor (e.g., Snapdragon 600 series, Apple A12, or equivalent) for smooth chart rendering.
- **Memory (RAM):** 3 GB or higher (To ensure the dashboard remains responsive without reloading tabs).
- **Operating System:**
 - **Android:** Android 10.0 or later.
 - **iOS:** iOS 14 or later.
- **Browser:**
 - **Android:** Google Chrome (latest version).
 - **iOS:** Safari (latest version).
- **Screen Resolution:** 720x1280 pixels or higher (The Bootstrap grid system will adapt the layout, but HD resolution is recommended for reading financial tables).
- **Internet Connection:** 4G/5G or stable Wi-Fi (Required for fetching real-time data via AJAX and syncing expenses to the cloud database).

7. SYSTEM DESIGN AND CODING

A. INTRODUCTION

Software design is a critical phase in the software engineering process, laying the foundation for all subsequent development activities. It transcends development paradigms and application areas, serving as a common thread in the creation of every engineered system. For the **Sports Management System**, the design phase was initiated after the tournament administration requirements were specified and analyzed. This step formed the core of the development phase, bridging the gap between the initial concept of "digital sports management" and the final **Python/Flask** implementation.

The central focus of this design is **data flexibility and operational speed**. Through the design process, the quality of the final dashboard is ensured by translating user requirements—such as dynamic scoring rules and conflict-free scheduling—into a structured framework. A robust design minimizes the risk of building rigid systems, ensuring that diverse sporting data (from cricket runs to weightlifting attempts) is handled securely and that the system remains maintainable for future sport additions via **JSON configuration schemas**.

In this phase, detailed representations of the **MySQL database schema**, Flask Blueprint structures (Admin, Manager, Public), and frontend interactive flows were refined, reviewed, and documented. From both technical and project management perspectives, system design plays a pivotal role in guiding the engineering process toward a successful, bug-free launch.

B. INPUT DESIGN

Input design is the process of converting user-oriented input into a computer-readable format. As a key component of the overall system design, input design for a sports application must be executed with meticulous attention to detail. This is because the collection of input data—specifically live match scores and player statistics—represents the most time-sensitive aspect of the system. Effective input design focuses on optimizing the methods of input collection to allow managers to update results from the field while ensuring the highest possible level of accuracy.

The primary objectives of input design for this project are as follows:

1. **Operational Efficiency:** Create an input process that is fast, using intuitive modals that allow managers to score a match without leaving the fixture list.
2. **Accuracy:** Achieve the highest possible accuracy in input data (e.g., validating that a "Snatch" weight is numeric) to prevent errors in tournament leaderboards.
3. **Adaptability:** Ensure that the input forms dynamically change based on the sport, showing "Wickets" for Cricket but "Goals" for Football.

C. Input Data Considerations:

When designing input data for the Sports Management System, the goal is to create a process that is easy, logical, and error-free. The data entry forms (managed via **HTML5, Jinja2, and jQuery**) are designed so that users are clear on the required fields. For example, the "Add Player" modal dynamically renders specific fields like "Batting Style" or "Weight Class" based on the team's sport to avoid ambiguity.

The input process involves the following stages in our context:

- **Data Recording:** Capturing raw tournament data (e.g., "100kg" for a lift or "4 runs" for a ball) from managers via web forms.
- **Data Validation:** Checking the accuracy of input data against predefined schemas (e.g., ensuring a player is not added to two teams in the same event).
- **Data Conversion:** Converting string inputs from forms into Python/JSON storage formats (e.g., serializing a cricket scorecard into a JSON object).
- **Data Transmission:** Securely transmitting data from the client browser to the Flask backend using **AJAX POST** requests to avoid page reloads.

D. Input Types in the Project:

A fundamental aspect of our input design is selecting data capture methods that reduce errors.

- **External Input:** Data sourced directly from the user, such as **Team Registration**, **Player Details**, and **Match Scores**.
- **Internal Input:** Data generated by the system, such as `event_id` associations and `created_at` timestamps for audit trails.
- **Interactive Input:** Input provided by managers in real-time, such as clicking the "Score" button on a fixture to open a modal and entering results which are immediately calculated.

E. OUTPUT DESIGN

Output design refers to the process of determining how the results of processing will be communicated to the users. In a sports application, outputs play a crucial role in conveying excitement and status, transforming raw scores into visual leaderboards and schedules. Designing

effective output involves organizing the presentation of information in a manner that is clear, actionable, and accessible to public spectators.

The outputs of the **Sports Management System** are categorized as follows:

1. **External Outputs:** Information presented to the public on the main website, such as the **"Live Fixtures"** list, **Team Squads**, and **Match Results**.
2. **Internal Outputs:** Logs generated within the system for debugging, such as database commit errors or invalid schema warnings.
3. **Interactive Outputs:** Dynamic tables (via Jinja2 loops) that respond to user queries, such as filtering events by "City" (e.g., Raipur) or "Sport" category.
4. **Turnaround Outputs:** Outputs used for further processing, such as the **"Total Score"** (Snatch + Clean & Jerk) calculated by the system, which determines the winner of a weightlifting session.

The key to designing effective outputs is ensuring that the right sporting metrics are presented in the right format. For instance, a Cricket result is best shown with "Runs/Wickets" and "Overs," while a Weightlifting result requires a detailed table of "Attempts." Outputs are structured to provide clear communication that facilitates tournament tracking.

The design of screens and interfaces plays a critical role. We utilize a **Responsive Dashboard Layout** (Sidebar + Main Content) for managers and a **Card-Based Layout** for public users. This ensures outputs are not only informative but also accessible on mobile devices for fans tracking games from the stands.

F. CONCLUSION

In the software engineering process, system design—including both input and output design—is foundational for building quality, reliable, and user-friendly sports administration tools. Input design ensures that dynamic sporting data is captured accurately and efficiently via adaptive forms, while output design guarantees that spectators and managers receive the match insights they need in a well-structured, visual manner.

These steps are critical for the success of the **Sports Management System**, as they directly impact both the functionality (scoring precision) and usability (ease of scheduling) of the final product. By focusing on these design principles, we ensure that the system meets the diverse needs of multi-sport tournaments, operates efficiently on the **Flask/MySQL** stack, and remains scalable for future enhancements.

8. DATABASE DESIGN

TABLE 1: Events

	Field	Type	Null	Key	Default	Extra
►	id	int	NO	PRI	NULL	auto_increment
	sport_id	int	NO	MUL	NULL	
	manager_id	int	NO	MUL	NULL	
	title	varchar(100)	NO		NULL	
	description	text	YES		NULL	
	start_date	datetime	NO		NULL	
	end_date	datetime	YES		NULL	
	status	varchar(20)	YES		NULL	
	venue_id	int	YES	MUL	NULL	
	rules_config	json	YES		NULL	
	round_name	varchar(50)	YES		NULL	

TABLE 2:Fixtures:

	Field	Type	Null	Key	Default	Extra
►	id	int	NO	PRI	NULL	auto_increment
	event_id	int	NO	MUL	NULL	
	venue_id	int	YES	MUL	NULL	
	start_time	datetime	NO		NULL	
	team_a_id	int	YES	MUL	NULL	
	team_b_id	int	YES	MUL	NULL	
	title	varchar(100)	YES		NULL	
	score_data	json	YES		NULL	

TABLE 3: Players:

	Field	Type	Null	Key	Default	Extra
►	id	int	NO	PRI	NULL	auto_increment
	team_id	int	NO	MUL	NULL	
	name	varchar(100)	NO		NULL	
	details	json	YES		NULL	

TABLE 4: Registrations:

	Field	Type	Null	Key	Default	Extra
►	user_id	int	NO	PRI	NULL	
	event_id	int	NO	PRI	NULL	

TABLE 5:Sports:

	Field	Type	Null	Key	Default	Extra
►	id	int	NO	PRI	NULL	auto_increment
	name	varchar(50)	YES	UNI	NULL	
	type	varchar(20)	YES		NULL	
	config_schema	json	NO		NULL	

TABLE 6: Teams:

	Field	Type	Null	Key	Default	Extra
►	id	int	NO	PRI	NULL	auto_increment
	event_id	int	NO	MUL	NULL	
	name	varchar(100)	NO		NULL	
	city	varchar(50)	YES		NULL	
	coach_name	varchar(100)	YES		NULL	
	captain_id	int	YES		NULL	

TABLE 6: Users:

	Field	Type	Null	Key	Default	Extra
►	id	int	NO	PRI	NULL	auto_increment
	username	varchar(50)	NO	UNI	NULL	
	email	varchar(120)	NO	UNI	NULL	
	password_hash	varchar(255)	NO		NULL	
	role	varchar(20)	YES		NULL	
	avatar	varchar(20)	YES		NULL	

TABLE 6: Venues:

	Field	Type	Null	Key	Default	Extra
►	id	int	NO	PRI	NULL	auto_increment
	name	varchar(100)	NO		NULL	
	city	varchar(50)	NO		NULL	
	address	varchar(200)	YES		NULL	

9.DATA FLOW DIAGRAM

It is a graphical tool used to describe and analyze the flow of data through a system. It focuses on the data flowing into the system, between processes, and in and out of data stores.

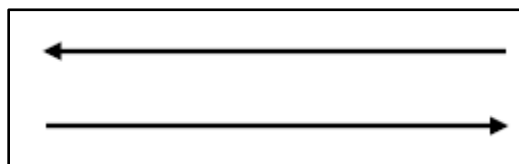
DFDs are of two types:

1. **Physical DFD:** The physical DFD is a model of the current system and is used to ensure that the current system has been clearly understood. It depicts *how* the system will be implemented (e.g., hardware, software, people).
2. **Logical DFD:** Logical DFDs are the model of the proposed system. They clearly show the requirements on which the new system should be built, focusing on *what* the system does rather than *how* it does it.

Notation Used In DFD:

There are four simple notations used to complete DFDs:

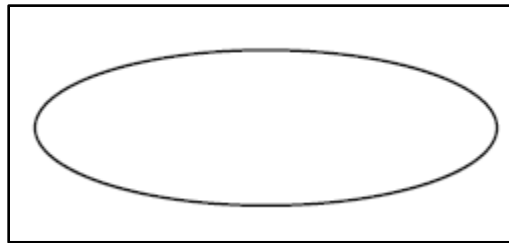
- **Dataflow:**



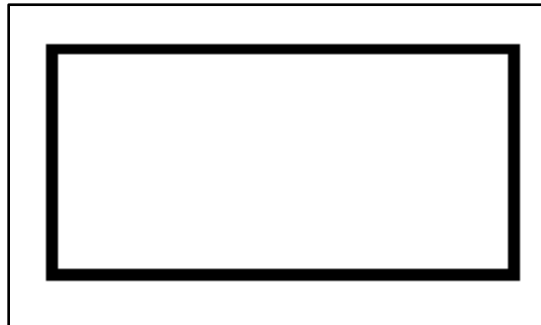
- **External Entity:**



- **Process:**

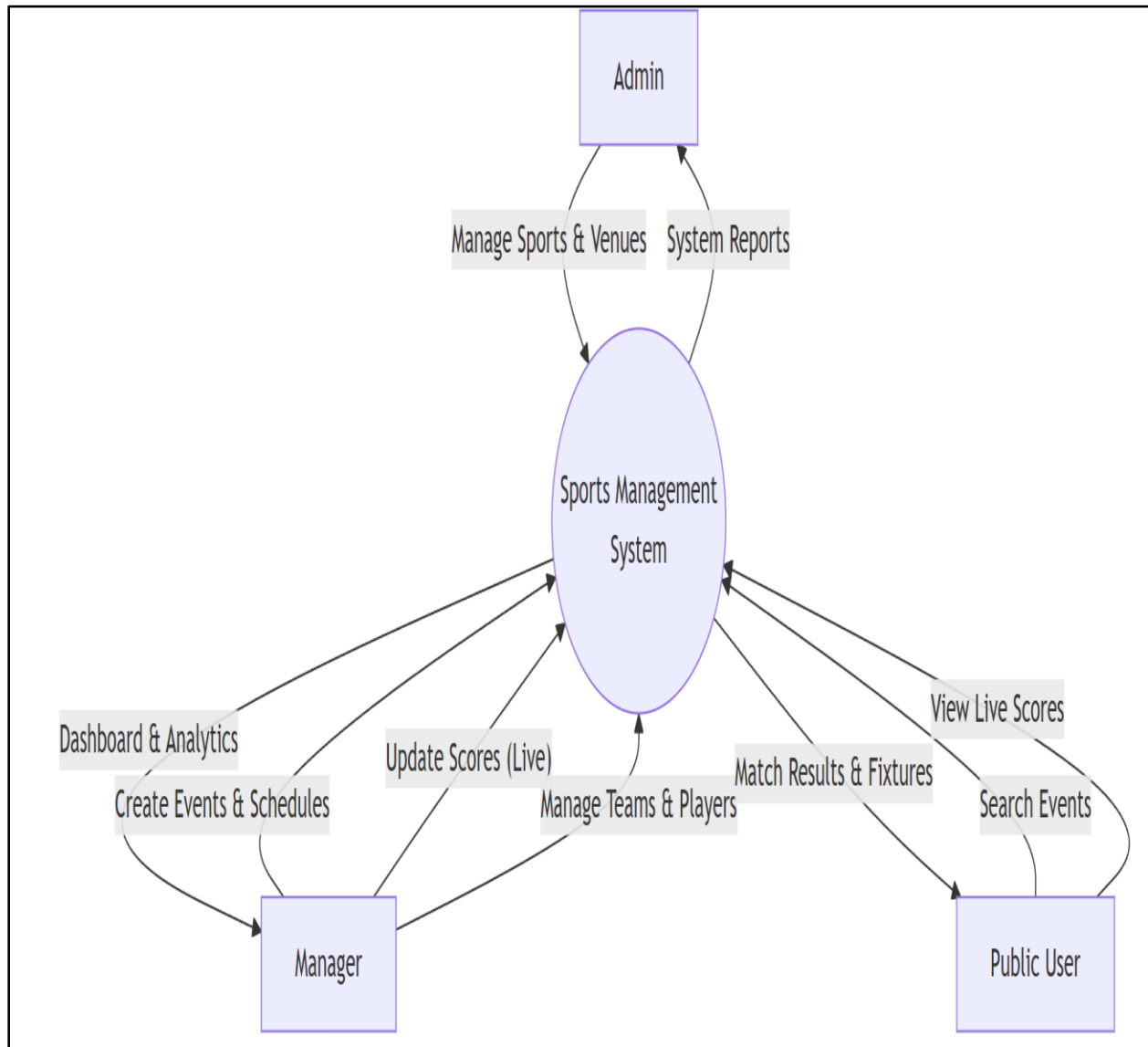


Data store:

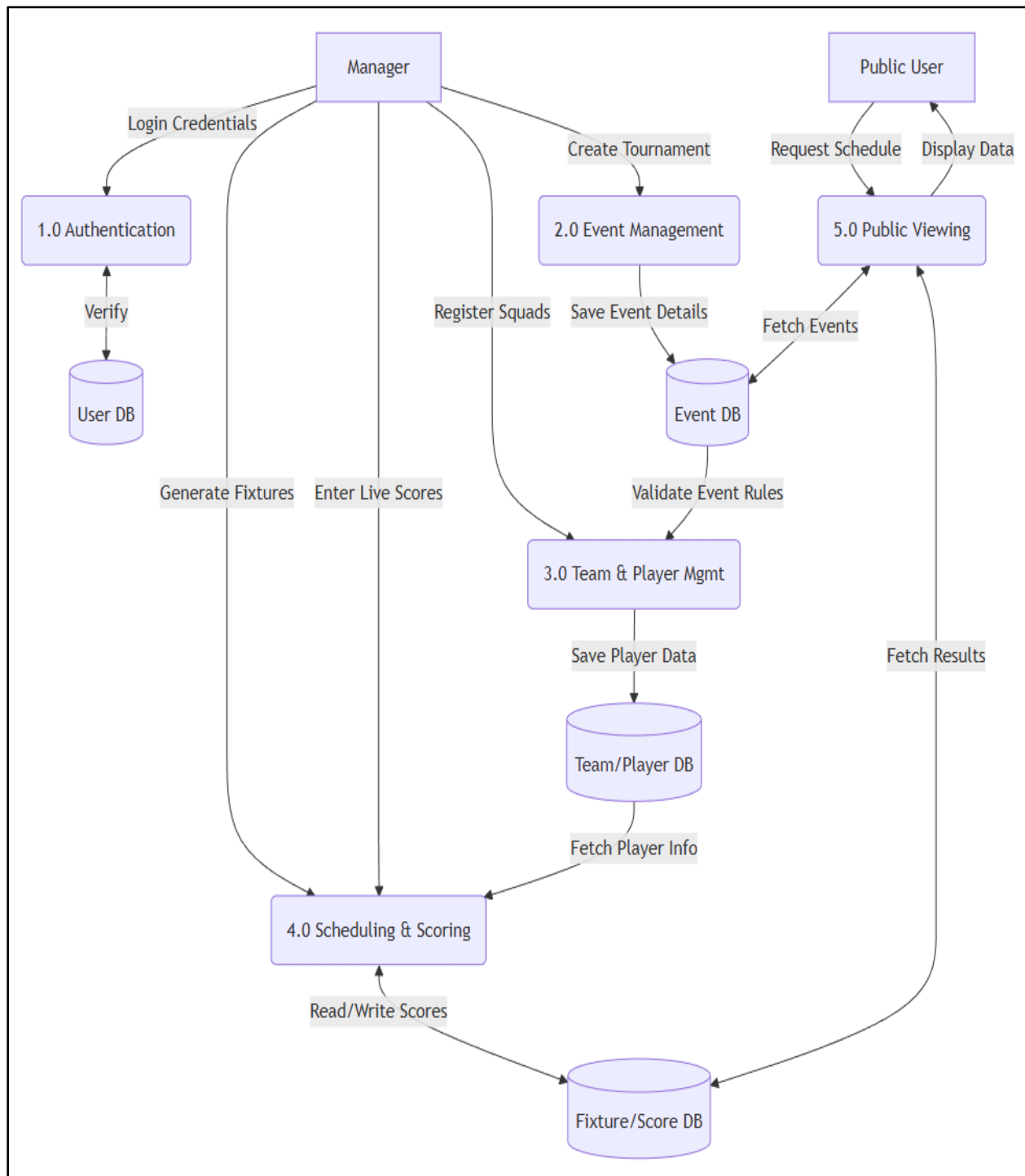


DFD FOR THE SYSTEM

- **Level 0 (Context Diagram):**



- **Level 1 (Detailed Diagram):**

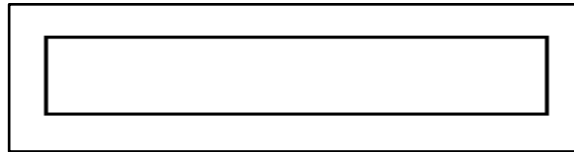


10.ENTITY RELATIONSHIP DIAGRAM

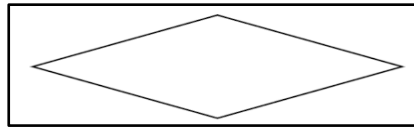
Entity Relationship Diagram (ERD) can express the overall logical structure of a database graphically. It shows the relationships between different data sets.

The components of E-R Diagram are:

- **Entity:**



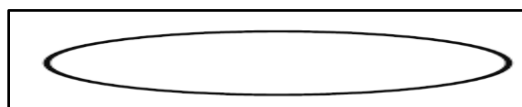
- **Relationship:**



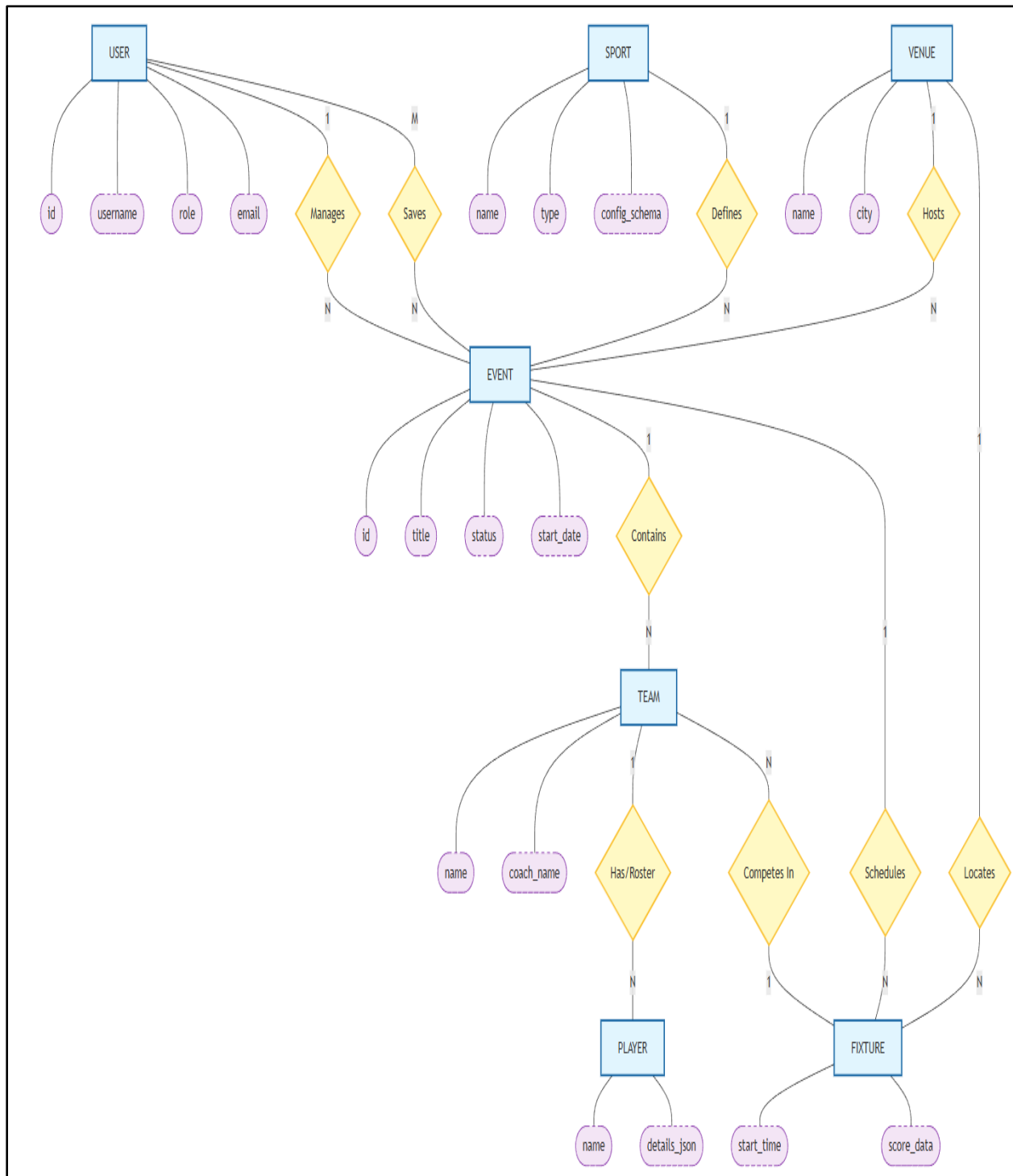
- **Weak Entity:**



- **Attributes:**

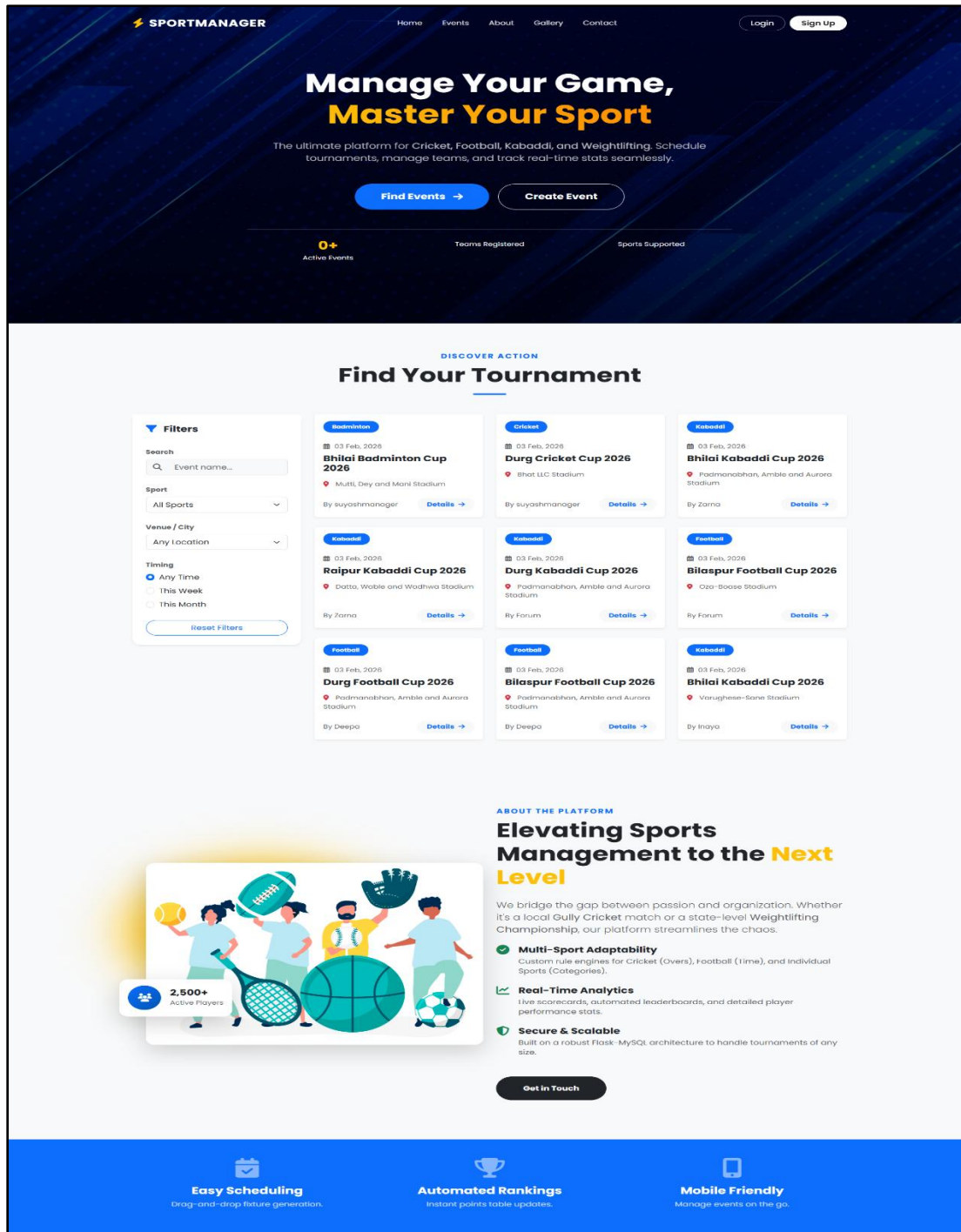


ENTITY RELATIONSHIP DIAGRAM



11.INPUT FORMS, PAGES AND CODING

Home Page:



Code

```
{% extends 'base.html' %}
{% block content %}
<section id="hero" class="d-flex align-items-center text-white"
  style="background: url({{ url_for('static', filename='images/hero-bg.jpg') }})
no-repeat center center/cover;">
  <div class="hero-overlay"></div>
  <div class="container position-relative z-2">
    <div class="row">
      <div class="col-lg-8 mx-auto text-center">
        <h1 class="display-3 fw-bold mb-4" data-aos="fade-down">
          Manage Your Game,<br> <span class="text-warning">Master Your
Sport</span>
        </h1>
        <div class="d-flex justify-content-center gap-3" data-aos="fade-up">
          <a href="#events" class="btn btn-primary btn-lg rounded-pill">Find
Events</a>
          {% if current_user.is_authenticated %}
            <a href="{{ url_for('admin.dashboard') }}" class="btn btn-
outline-light btn-lg rounded-pill">Go to Dashboard</a>
          {% else %}
            <a href="{{ url_for('auth.register') }}" class="btn btn-outline-
light btn-lg rounded-pill">Create Event</a>
          {% endif %}
        </div>
        <div class="row mt-5 pt-4 border-top border-light border-opacity-
25">
          <div class="col-4"><h3 class="fw-bold text-warning">{{
stats.events }}</h3><small>Active Events</small></div>
          <div class="col-4"><h3 class="fw-bold text-warning">{{
stats.teams }}</h3><small>Teams</small></div>
          <div class="col-4"><h3 class="fw-bold text-warning">{{
stats.sports }}</h3><small>Sports</small></div>
        </div>
      </div>
    </div>
  </div>
</section>
<section id="events" class="py-5 bg-light position-relative">
  <div class="container py-4">
    <div class="text-center mb-5"><h2 class="fw-bold display-5">Find Your
Tournament</h2></div>
    <div class="row g-4">
      <div class="col-lg-3" data-aos="fade-right">
        <div class="card border-0 shadow-sm rounded-4 sticky-top"
style="top: 100px;">
          <div class="card-body p-4">
            <form id="filterForm">
```



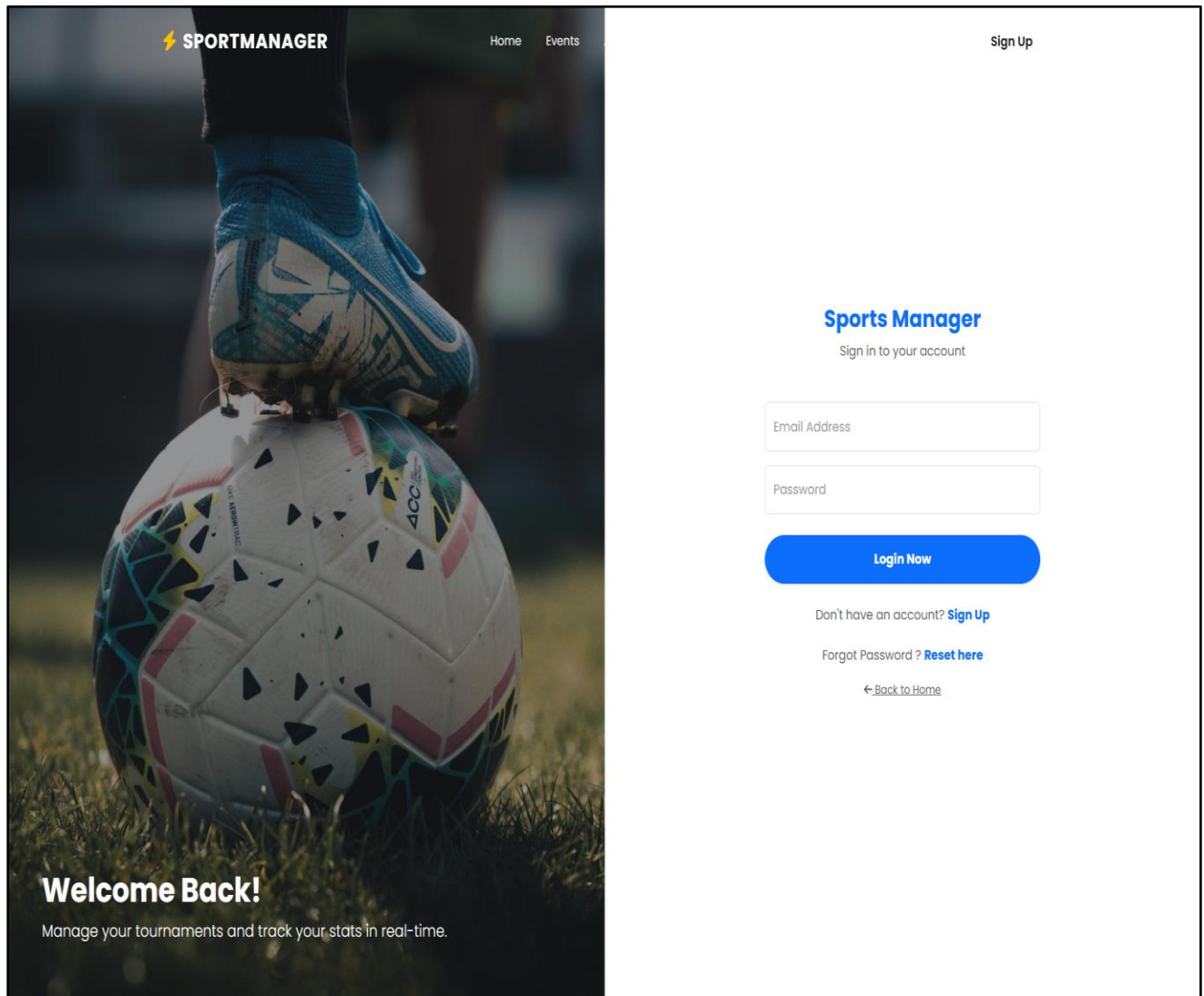
```

        </div>
    </div>
</div>
</div>
</section>
<section id="gallery" class="py-5 bg-light">
    <div class="container py-5">
        <div class="text-center mb-5"><h2 class="fw-bold display-5">Our
Gallery</h2></div>
        <div class="row g-3">
            <div class="col-lg-6"></div>
            <div class="col-lg-6">
                </div>
            </div>
        </div>
</section>
<section id="contact" class="py-5 position-relative">
    <div class="container py-5">
        <div class="row g-5">
            <div class="col-lg-5">
                <h2 class="display-5 fw-bold mb-4">Have Questions?</h2>
                <div class="d-flex mb-4">
                    <i class="fas fa-envelope fa-lg me-3"></i>
                    <div><h5 class="fw-bold">Email Us</h5><p class="text-
muted">support@sportsmanager.com</p></div>
                </div>
            </div>
            <div class="col-lg-7">
                <div class="card border-0 shadow-lg rounded-4 p-4">
                    <form>
                        <div class="row g-3">
                            <div class="col-md-6"><input type="text" class="form-
control" placeholder="Name"></div>
                            <div class="col-md-6"><input type="email" class="form-
control" placeholder="Email"></div>
                            <div class="col-12"><textarea class="form-control"
placeholder="Message" style="height: 150px"></textarea></div>
                            <button type="submit" class="btn btn-primary btn-lg rounded-
pill w-100">Send Message</button>
                        </div>
                    </form>
                </div>
            </div>
        </div>
    </div>
</section>

```

```
<section class="py-5 bg-dark text-white text-center">
  <div class="container py-4">
    <h2 class="fw-bold mb-3">Ready to Start Your League?</h2>
    <a href="{ { url_for('auth.register') } }" class="btn btn-warning btn-lg
rounded-pill px-5">Create Free Account</a>
  </div>
</section>
{% include 'partials/global_event_modal.html' %}
{% endblock %}
```

Login Page:



Code:

```
{% extends "base.html" %}
{% block content %}
<div class="container-fluid vh-100 p-0">
  <div class="row g-0 h-100">
    <div class="col-lg-6 d-none d-lg-block position-relative">
      
      <div class="position-absolute top-0 start-0 w-100 h-100 bg-dark opacity-50"></div>
```



```

        <div class="position-absolute bottom-0 start-0 p-5 text-white">
            <h1 class="fw-bold">Welcome Back!</h1>
            <p class="lead">Manage your tournaments and track your stats in real-
time.</p>
        </div>
    </div>
    <div class="col-lg-6 d-flex align-items-center justify-content-center bg-
white">
        <div class="w-100 p-5" style="max-width: 500px;">
            <div class="text-center mb-5"><h3 class="fw-bold text-
primary">Sports Manager</h3><p class="text-muted">Sign in to your
account</p></div>
            <form method="POST" action="">
                {{ form.hidden_tag() }}
                <div class="form-floating mb-3">
                    {{ form.email(class="form-control",
placeholder="name@example.com") }} {{ form.email.label }}
                    {% for error in form.email.errors %}<div class="text-danger
small">{{ error }}</div>{% endfor %}
                </div>
                <div class="form-floating mb-4">
                    {{ form.password(class="form-control",
placeholder="Password") }} {{ form.password.label }}
                    {% for error in form.password.errors %}<div class="text-danger
small">{{ error }}</div>{% endfor %}
                </div>
                <button type="submit" class="btn btn-primary w-100 py-3 rounded-
pill fw-bold shadow-sm">Login Now</button>
            </form>
            <div class="text-center mt-4"><span class="text-muted">Don't have
an account?</span> <a href="{{ url_for('auth.register') }}" class="text-primary
fw-bold text-decoration-none">Sign Up</a></div>
            <div class="text-center mt-4"><span class="text-muted">Forgot
Password?</span> <a href="{{ url_for('auth.reset_request') }}" class="text-
primary fw-bold text-decoration-none">Reset here</a></div>
            <div class="text-center mt-3"><a href="{{ url_for('public.index') }}"
class="small text-muted"><i class="fas fa-arrow-left"></i> Back to
Home</a></div>
        </div>
    </div>
</div>
{% endblock %}

```

Register Page:

SPORTMANAGER Home About Us Contact Us

[Login](#) [Sign Up](#)

Create Account

Username

Email Address

Password Confirm Password

I want to...
Join/View Events

Managers can create tournaments. Viewers can join teams.

[Sign Up](#)

Already registered? [Login here](#)

Forgot Password? [Reset here](#)

Code:

```
{% extends "base.html" %}
{% block content %}
<div class="container-fluid min-vh-100 p-0">
  <div class="row g-0 min-vh-100">
    <div class="col-lg-5 d-none d-lg-block position-relative">
      
      <div class="position-absolute top-0 start-0 w-100 h-100 bg-primary opacity-50"></div>
      <div class="position-absolute center-0 start-0 p-5 text-white top-50 translate-middle-y">
        <h1 class="display-4 fw-bold">Join the League</h1><p
```

```

class="lead">Create events, build teams, and compete.</p>
</div>
</div>
<div class="col-lg-7 d-flex align-items-center justify-content-center bg-
light">
<div class="w-100 p-4 p-md-5" style="max-width: 600px;">
<div class="card border-0 shadow-lg rounded-4 overflow-hidden">
<div class="card-body p-5">
<h3 class="fw-bold mb-4 text-center">Create Account</h3>
<form method="POST" action="">
{{ form.hidden_tag() }}
<div class="row g-3">
<div class="col-12"><div class="form-floating">{{
form.username(class="form-control", placeholder="Username") }} {{
form.username.label }} {% for error in form.username.errors %}<span
class="text-danger small">{{ error }}</span>{% endfor %}</div></div>
<div class="col-12"><div class="form-floating">{{
form.email(class="form-control", placeholder="Email") }} {{ form.email.label
}} {% for error in form.email.errors %}<span class="text-danger small">{{ error
}}</span>{% endfor %}</div></div>
<div class="col-md-6"><div class="form-floating">{{
form.password(class="form-control", placeholder="Password") }} {{
form.password.label }} {% for error in form.password.errors %}<span
class="text-danger small">{{ error }}</span>{% endfor %}</div></div>
<div class="col-md-6"><div class="form-floating">{{
form.confirm_password(class="form-control", placeholder="Confirm") }} {{
form.confirm_password.label }} {% for error in form.confirm_password.errors
%}<span class="text-danger small">{{ error }}</span>{% endfor
%}</div></div>
<div class="col-12"><div class="form-floating">{{
form.role(class="form-select") }} {{ form.role.label }}</div><div class="form-
text small mt-1"><i class="fas fa-info-circle"></i> Managers can create
tournaments. Viewers can join teams.</div></div>
<div class="col-12 mt-4"><button type="submit" class="btn
btn-primary w-100 py-3 rounded-pill fw-bold">Sign Up</button></div>
</div>
</form> <div class="text-center mt-4"><span class="text-
muted">Already registered?</span> <a href="{{ url_for('auth.login') }}"
class="text-primary fw-bold">Login here</a></div>
<div class="text-center mt-4"><span class="text-muted">Forgot
Password?</span> <a href="{{ url_for('auth.reset_request') }}" class="text-
primary fw-bold text-decoration-none">Reset here</a></div> </div>
</div>
</div>
</div>
</div>
{{ % endblock %}}

```

Manager OverView Tab:

 **MANAGER
PANEL**

 Overview

 Manage Tournaments

 Create New Event

 My Profile

 Logout

Manager Overview

2
Events



12
Teams



Recent Tournaments

#	Event Name	Sport	Date	Status	Actions
1	Bhilai Badminton Cup 2026	Badminton	03 Feb 2026	completed	Manage
2	Durg Cricket Cup 2026	Cricket	03 Feb 2026	completed	Manage

© 2026 Sports Manager. All rights reserved.

Code:

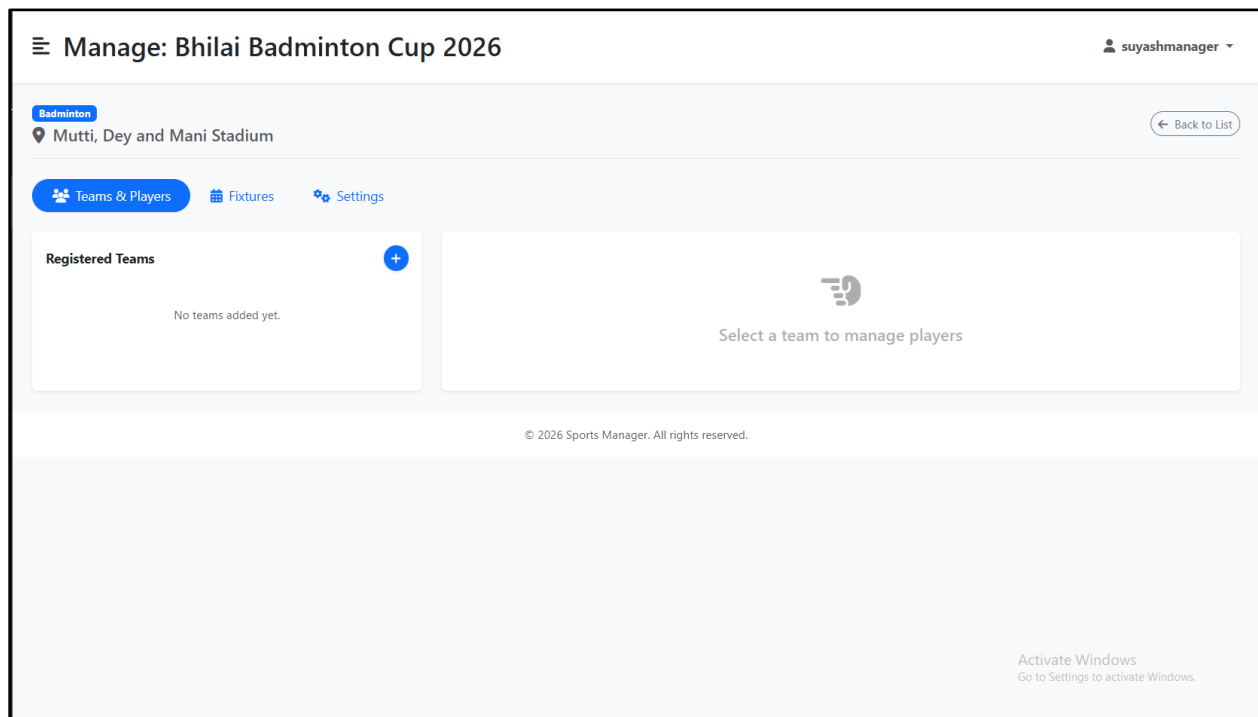
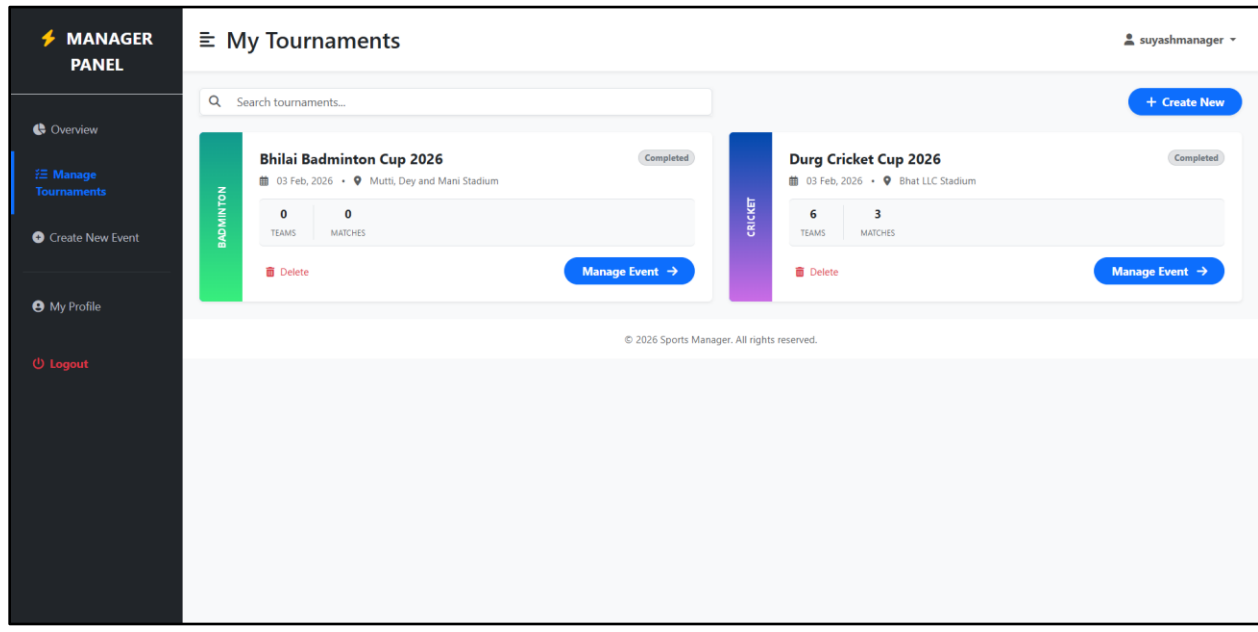
```
{% extends "layouts/dashboard_base.html" %}
{% block page_title %}Manager Overview{% endblock %}
{% block content %}
<div class="row g-3 my-2">
```

```

<div class="col-md-3">
  <div class="p-3 bg-white shadow-sm d-flex justify-content-around align-items-center
rounded">
    <div><h3 class="fs-2">{{ event_count }}</h3><p class="fs-5">Events</p></div>
    <i class="fas fa-trophy fs-1 primary-text border rounded-full secondary-bg p-3"></i>
  </div>
</div>
<div class="col-md-3">
  <div class="p-3 bg-white shadow-sm d-flex justify-content-around align-items-center
rounded">
    <div><h3 class="fs-2">12</h3> <p class="fs-5">Teams</p></div>
    <i class="fas fa-users fs-1 primary-text border rounded-full secondary-bg p-3"></i>
  </div>
</div>
</div>
<div class="row my-5">
  <h3 class="fs-4 mb-3">Recent Tournaments</h3>
  <div class="col">
    <div class="table-responsive bg-white rounded shadow-sm">
      <table class="table table-hover mb-0">
        <thead class="bg-light">
          <tr>
            <th scope="col" width="50">#</th><th scope="col">Event Name</th><th
scope="col">Sport</th><th scope="col">Date</th><th scope="col">Status</th><th
scope="col">Actions</th>
          </tr>
        </thead>
        <tbody>
          {% for event in events %}
          <tr> <th scope="row">{{ loop.index }}</th> <td>{{ event.title }}</td>
            <td>{{ event.sport.name }}</td>
            <td>{{ event.start_date.strftime('%d %b %Y') }}</td>
            <td><span class="badge bg-{{ 'success' if event.status == 'upcoming' else
'secondary' }}">{{ event.status }}</span></td>
            <td><a href="{{ url_for('users.manage_event', event_id=event.id) }}" class="btn
btn-sm btn-outline-primary">Manage</a></td>
          </tr>
          {% else %}
          <tr><td colspan="6" class="text-center py-4">No events created yet.</td></tr>
          {% endfor %}
        </tbody>
      </table>
    </div>
  </div>
</div>
{% endblock %}

```

Manager Manage Tournament Tab:



Manage: Bhilai Badminton Cup 2026

suyashmanager

Badminton

Mutti, Dey and Mani Stadium

Back to List

Teams & Players

Fixtures

Settings

Schedule Match

Round / Phase

Group Stage

Date & Time

dd-mm-yyyy --:-- --

Session Title

e.g. Men's 85kg Group A

Add to Schedule

Upcoming Schedule

DATE	MATCH DETAILS	ROUND	STATUS
No matches scheduled yet.			

© 2026 Sports Manager. All rights reserved.

Activate Windows
Go to Settings to activate Windows.

Manage: Bhilai Badminton Cup 2026

suyashmanager

Badminton

Mutti, Dey and Mani Stadium

Back to List

Teams & Players

Fixtures

Settings

Event Configuration

CURRENT STATUS

Upcoming

Live Now

Completed

Event Title

Bhilai Badminton Cup 2026

Description

Deleniti aut sed nam recusandae officia.

Delete Event

Save Changes

Activate Windows
Go to Settings to activate Windows.

Code:

```
{% extends "layouts/dashboard_base.html" %}
{% block page_title %}Manage: {{ event.title }}{% endblock %}
{% block content %}
<div class="d-flex justify-content-between align-items-center mb-4 border-bottom pb-3">
  <div><span class="badge bg-primary mb-1">{{ event.sport.name }}</span><h5 class="text-
muted mb-0"><i class="fas fa-map-marker-alt me-1"></i> {{ event.venue.name if event.venue
else "TBD" }}</h5></div>
  <a href="{{ url_for('users.manager_my_events') }}" class="btn btn-outline-secondary rounded-
pill btn-sm"><i class="fas fa-arrow-left me-1"></i> Back to List</a>
</div>
<ul class="nav nav-pills mb-4" id="pills-tab" role="tablist">
  <li class="nav-item"><button class="nav-link active rounded-pill px-4" id="pills-teams-tab"
data-bs-toggle="pill" data-bs-target="#pills-teams"><i class="fas fa-users me-
2"></i> Teams</button></li>
  <li class="nav-item"><button class="nav-link rounded-pill px-4" id="pills-fixtures-tab" data-
bs-toggle="pill" data-bs-target="#pills-fixtures"><i class="fas fa-calendar-alt me-
2"></i> Fixtures</button></li>
  <li class="nav-item"><button class="nav-link rounded-pill px-4" id="pills-settings-tab" data-
bs-toggle="pill" data-bs-target="#pills-settings"><i class="fas fa-cogs me-
2"></i> Settings</button></li>
</ul>
<div class="tab-content" id="pills-tabContent">
  <div class="tab-pane fade show active" id="pills-teams">
    <div class="row">
      <div class="col-lg-4">
        <div class="card border-0 shadow-sm h-100">
          <div class="card-header bg-white d-flex justify-content-between align-items-center
py-3">
            <h6 class="fw-bold mb-0">Registered Teams</h6>
            <button class="btn btn-sm btn-primary rounded-circle shadow-sm" data-bs-
toggle="modal" data-bs-target="#addTeamModal"><i class="fas fa-plus"></i></button>
          </div>
          <div class="list-group list-group-flush" id="teamList">
            {% for team in teams %}
              <a href="#" class="list-group-item list-group-item-action py-3 team-item"
onclick="loadSquad({{ team.id }}, '{{ team.name }}')">
                <div class="d-flex w-100 justify-content-between"><h6 class="mb-1 fw-
bold">{{ team.name }}</h6><small class="text-muted">{{ team.city }}</small></div>
                <small class="text-primary"><i class="fas fa-user-tie me-1"></i> {{
team.coach_name }}</small>
              </a>
            {% else %}
              <div class="text-center py-4 text-muted"><small>No teams added
```



```

yet.</small></div>
    {% endfor %}
</div>
</div>
<div class="col-lg-8">
    <div class="card border-0 shadow-sm h-100" id="squadPanel" style="display:none;">
        <div class="card-header bg-white py-3 d-flex justify-content-between align-items-
center">
            <div><h5 class="fw-bold mb-0" id="selectedTeamName">Select a
Team</h5><small class="text-muted">Squad Management</small></div>
            <button class="btn btn-outline-primary btn-sm rounded-pill fw-bold"
onclick="openAddPlayerModal()"><i class="fas fa-user-plus me-1"></i> Add Player</button>
            </div>
            <div class="card-body bg-light" id="squadListContainer"><div class="text-center
py-5"><div class="spinner-border text-primary"></div></div></div>
            </div>
            <div class="card border-0 shadow-sm h-100 d-flex align-items-center justify-content-
center p-5" id="squadEmptyState">
                <div class="text-center text-muted opacity-50"><i class="fas fa-hand-point-left fa-3x
mb-3"></i><h5>Select a team</h5></div>
                </div>
            </div>
        </div>
    </div>
<div class="tab-pane fade" id="pills-fixtures">
    <div class="row">
        <div class="col-lg-4 mb-4">
            <div class="card border-0 shadow-sm">
                <div class="card-header bg-primary text-white py-3"><h6 class="fw-bold mb-
0">Schedule Match</h6></div>
                <div class="card-body">
                    <form id="addFixtureForm">
                        <div class="mb-3"><label class="form-label small fw-
bold">Round</label><select name="round_name" class="form-select"><option>Group
Stage</option><option>Final</option></select></div>
                        <div class="mb-3"><label class="form-label small fw-
bold">Date</label><input type="datetime-local" name="start_time" class="form-control"
required></div>
                        {% if event.sport.type == 'team' %}
                        <div class="mb-3 p-3 bg-light rounded border">
                            <select name="team_a" class="form-select mb-2" required><option value=""
disabled selected>Home Team</option>{% for team in teams %}<option value="{{ team.id
}}">{{ team.name }}</option>{% endfor %}</select>
                            <div class="text-center small fw-bold mb-2">VS</div>
                            <select name="team_b" class="form-select" required><option value=""

```

```

disabled selected>Away Team</option>{% for team in teams %}<option value="{{ team.id
}}">{{ team.name }}</option>{% endfor %}</select>
</div>
<input type="hidden" name="title" value="Match">
{% else %}
<div class="mb-3"><label class="form-label small fw-bold">Session
Title</label><input type="text" name="title" class="form-control" required></div>
{% endif %}
<button type="submit" class="btn btn-primary w-100 rounded-pill fw-
bold">Add to Schedule</button>
</form>
</div>
</div>
</div>
<div class="col-lg-8">
<div class="card border-0 shadow-sm h-100">
<div class="card-header bg-white py-3"><h6 class="fw-bold mb-0">Upcoming
Schedule</h6></div>
<div class="card-body p-0">
<div class="table-responsive">
<table class="table table-hover align-middle mb-0">
<thead class="bg-light small text-uppercase"><tr><th class="ps-
4">Date</th><th>Details</th><th>Round</th><th class="text-end pe-
4">Status</th></tr></thead>
<tbody id="fixtureTableBody">
{% for fixture in event.fixtures %}
<tr>
<td class="ps-4 small"><div class="fw-bold">{{
fixture.start_time.strftime('%d %b') }}</div>{{ fixture.start_time.strftime('%I:%M %p') }}</td>
<td>{% if event.sport.type == 'team' %}<span class="fw-bold">{{
fixture.team_a_obj.name }}</span> vs <span class="fw-bold">{{ fixture.team_b_obj.name
}}</span>{% else %}{{ fixture.title }}{% endif %}</td>
<td><span class="badge bg-light text-dark border">{{
fixture.round_name }}</span></td>
<td class="text-end pe-4"><button class="btn btn-sm btn-outline-
primary rounded-pill px-3">Score <i class="fas fa-edit ms-1"></i></button></td>
</tr>
{% else %}
<tr><td colspan="4" class="text-center py-5 text-muted">No matches
scheduled.</td></tr>
{% endfor %}
</tbody>
</table>
</div>
</div>
</div>

```

```

        </div>
    </div>
</div>
<div class="tab-pane fade" id="pills-settings">
    <div class="row justify-content-center">
        <div class="col-lg-8">
            <div class="card border-0 shadow-sm rounded-4">
                <div class="card-body p-5">
                    <h5 class="fw-bold pb-3 border-bottom mb-4">Event Configuration</h5>
                    <form action="{{ url_for('users.update_event_settings', event_id=event.id) }}"
method="POST">
                        <div class="row g-4">
                            <div class="col-md-12">
                                <label class="form-label fw-bold small text-muted">Status</label>
                                <div class="d-flex gap-2">
                                    <input type="radio" class="btn-check" name="status" id="s1"
value="upcoming" {{ 'checked' if event.status == 'upcoming' }}><label class="btn btn-outline-
primary rounded-pill" for="s1">Upcoming</label>
                                    <input type="radio" class="btn-check" name="status" id="s2"
value="live" {{ 'checked' if event.status == 'live' }}><label class="btn btn-outline-danger
rounded-pill" for="s2">Live</label>
                                    <input type="radio" class="btn-check" name="status" id="s3"
value="completed" {{ 'checked' if event.status == 'completed' }}><label class="btn btn-outline-
secondary rounded-pill" for="s3">Completed</label>
                                </div>
                            </div>
                            <div class="col-md-12"><label class="form-label small fw-
bold">Title</label><input type="text" name="title" class="form-control" value="{{ event.title
}}"></div>
                            <div class="col-md-12"><label class="form-label small fw-
bold">Description</label><textarea name="description" class="form-control" rows="4">{{
event.description }}</textarea></div>
                        </div>
                        <div class="mt-5 d-flex justify-content-between"><button type="button"
class="btn btn-outline-danger border-0">Delete</button><button type="submit" class="btn btn-
success rounded-pill px-5 fw-bold">Save Changes</button></div>
                    </form>
                </div>
            </div>
        </div>
    </div>
</div>
</div>
<div class="modal fade" id="addTeamModal" tabindex="-1">
    <div class="modal-dialog modal-dialog-centered">
        <div class="modal-content border-0 shadow">

```

```

        <div class="modal-header border-0"><h5 class="modal-title fw-bold">Add New
Team</h5><button type="button" class="btn-close" data-bs-dismiss="modal"></button></div>
        <div class="modal-body">
            <form id="addTeamForm">
                <div class="mb-3"><label>Team Name</label><input type="text" name="name"
class="form-control" required></div>
                <div class="row"><div class="col-6 mb-3"><label>City</label><input type="text"
name="city" class="form-control" required></div><div class="col-6 mb-
3"><label>Coach</label><input type="text" name="coach_name" class="form-control"
required></div></div>
                <button type="submit" class="btn btn-primary w-100 rounded-pill">Save
Team</button>
            </form>
        </div>
    </div>
</div>
<div class="modal fade" id="addPlayerModal" tabindex="-1">
    <div class="modal-dialog modal-dialog-centered">
        <div class="modal-content border-0 shadow">
            <div class="modal-header border-0"><h5 class="modal-title fw-bold">Add
Player</h5><button type="button" class="btn-close" data-bs-dismiss="modal"></button></div>
            <div class="modal-body">
                <form id="addPlayerForm">
                    <input type="hidden" id="currentTeamId">
                    <div class="mb-3"><label>Full Name</label><input type="text" id="playerName"
class="form-control" required></div>
                    <hr class="opacity-25 my-3"><h6 class="small fw-bold text-muted">Sport
Specifics</h6>
                    <div id="dynamicPlayerFields" class="row g-2"></div>
                    <div class="mt-4"><button type="button" class="btn btn-success w-100 rounded-
pill" onclick="submitPlayer()">Add to Squad</button></div>
                </form>
            </div>
        </div>
    </div>
</div>
<div class="modal fade" id="scoreModal" tabindex="-1">
    <div class="modal-dialog modal-xl">
        <div class="modal-content">
            <div class="modal-header bg-light"><h5 class="modal-title fw-bold"
id="scoreModalTitle">Score Match</h5><button type="button" class="btn-close" data-bs-
dismiss="modal"></button></div>
            <div class="modal-body p-0">
                <form id="scoreForm"><input type="hidden" id="scoreFixtureId" name="fixture_id">
                <div class="table-responsive"><table class="table table-bordered text-center mb-

```

```

0"><thead class="bg-white"><tr><th rowspan="2">Player</th><th colspan="3">Snatch</th><th colspan="3">Clean & Jerk</th><th rowspan="2">Total</th></tr><tr><th>1</th><th>2</th><th>3</th><th>1</th><th>2</th><th>3</th></tr></thead><tbody id="scoreTableBody"></tbody></table></div>
</form>
</div>
<div class="modal-footer"><button type="button" class="btn btn-primary"
onclick="submitScores()">Save Results</button></div>
</div>
</div>
<div>
{% endblock %}
{% block scripts %}
<script>
const SPORT_SCHEMA = {{ sport_schema|tojson|default('{}') }};
let activeTeamId = null;
$(document).ready(function() {
    $('#addTeamForm').submit(function(e) { e.preventDefault(); $.post("/users/api/event/{ {
event.id } }/add_team", $(this).serialize(), function() { location.reload(); }); });
    $('#addFixtureForm').submit(function(e) { e.preventDefault(); $.post("/users/api/event/{ {
event.id } }/add_fixture", $(this).serialize(), function() { location.reload(); }); });
    function loadSquad(teamId, teamName) {
        activeTeamId = teamId;
        $('#squadEmptyState').hide(); $('#squadPanel').show();
        $('#selectedTeamName').text(teamName); $('#currentTeamId').val(teamId);
        $('#squadListContainer').html('<div class="text-center py-5"><div class="spinner-border
text-primary"></div></div>');
        $.get("/users/api/team/" + teamId + "/get_players", function(resp) {
        $('#squadListContainer').html(resp.html); });
        function openAddPlayerModal() {
            if(!activeTeamId) return alert("Select a team first.");
            let container = $('#dynamicPlayerFields').empty();
            if(SPORT_SCHEMA.roles?.length) container.append('<div class="col-12 mb-2"><label
class="small">Role</label><select class="form-select"
id="field_role">${ SPORT_SCHEMA.roles.map(r=>`<option>${r}</option>`).join("")}</select><
/div>`);
            if(SPORT_SCHEMA.categories) {
                container.append('<div class="col-6 mb-2"><label class="small">Weight
Class</label><select class="form-select"
id="field_weight_class">${ SPORT_SCHEMA.categories.map(c=>`<option>${c}</option>`).join
("")}</select></div><div class="col-6 mb-2"><label class="small">Body Weight</label><input
type="number" step="0.1" class="form-control" id="field_body_weight"></div>`);
            } else container.append('<div class="col-12 mb-2"><label class="small">Jersey
No</label><input type="number" class="form-control" id="field_jersey"></div>`);
            $('#addPlayerModal').modal('show');
        }
    }
}

```

```

function submitPlayer() {
    let payload = { name: $('#playerName').val() };
    $('[id^="field_"]').each(function() { payload[$(this).attr('id').replace('field_', '')] =
$(this).val(); });
    $.ajax({ url: "/users/api/team/" + activeTeamId + "/add_player", type: "POST", contentType:
"application/json", data: JSON.stringify(payload), success: function() {
$('#addPlayerModal').modal('hide'); loadSquad(activeTeamId, $('#selectedTeamName').text());
}});
}
function openScoreModal(fixtureId, title) {
    $('#scoreModalTitle').text("Scoring: " + title); $('#scoreFixtureId').val(fixtureId);
    $('#scoreTableBody').html('<tr><td colspan="9" class="text-center"><div class="spinner-
border text-primary"></div></td></tr>');
    $.get(`/users/api/fixture/${fixtureId}/get_scores`, function(resp) {
        let html = resp.players.map(p => {
            let s = p.scores || {};
            return `<tr><td class="text-start ps-3 fw-bold">${p.name}</td>
<td><input type="number" class="form-control form-control-sm"
name="p_${p.id}_s1" value="${s.s1}"></td><td><input type="number" class="form-control
form-control-sm" name="p_${p.id}_s2" value="${s.s2}"></td><td><input type="number"
class="form-control form-control-sm" name="p_${p.id}_s3" value="${s.s3}"></td>
<td><input type="number" class="form-control form-control-sm"
name="p_${p.id}_c1" value="${s.c1}"></td><td><input type="number" class="form-control
form-control-sm" name="p_${p.id}_c2" value="${s.c2}"></td><td><input type="number"
class="form-control form-control-sm" name="p_${p.id}_c3" value="${s.c3}"></td>
<td class="fw-bold bg-light">${s.total}</td></tr>`;
        }).join("");
    $('#scoreTableBody').html(html || '<tr><td colspan="9">No players found.</td></tr>');
    $('#scoreModal').modal('show');
});
}
function submitScores() { $.post("/users/api/save_scores", $('#scoreForm').serialize(),
function(resp) { alert(resp.message); $('#scoreModal').modal('hide'); }); }
function deletePlayer(pid) { if(confirm("Remove player?"))
$.post(`/users/api/player/${pid}/delete`, function() { loadSquad(activeTeamId,
$('#selectedTeamName').text()); }); }
</script>
{% endblock %}

```

Manager Create Tournament Tab:

The screenshot shows the 'Manager Create Tournament Tab' interface. On the left is a dark sidebar with the 'MANAGER PANEL' header and navigation links: 'Overview', 'Manage Tournaments', 'Create New Event' (highlighted), 'My Profile', and 'Logout'. The main area is titled 'Event Management' and shows a modal for 'Step 1: The Basics'. The modal has a close button (X) and a subtitle 'Define the core details of the event.' It contains the following fields: 'Tournament Name' (text input), 'Sport' (dropdown menu with 'Select...' text), 'Venue' (dropdown menu with 'To Be Decided' text), 'Start Date' (calendar icon and 'dd-mm-yyyy' text), and 'Description (Optional)' (text area). At the bottom of the modal are 'Cancel' and 'Next →' buttons. A footer note at the bottom center reads '© 2020 Sports Manager. All rights reserved.'

The screenshot shows the 'Step 2: Configuration' modal. It has a close button (X) and a subtitle 'Set squad limits and sport specifics.' The modal is divided into two sections: 'Participation Details' and 'Game Rules'. The 'Participation Details' section includes 'Max Teams / Participants' (input: 8), 'Squad Size (Max Players)' (input: 15), and 'Min Players Required' (input: 11). The 'Game Rules' section includes 'Match Duration (mins)' (input: 40) and 'Weight Limit (kg)' (input: 85). At the bottom right are 'Back' and 'Next →' buttons.

Step 3: Custom Rules

×

Add specific regulations for this tournament.

RULE LABEL	VALUE / CONSTRAINT	DESCRIPTION (OPTIONAL)
+ Add Custom Rule		

Back

Launch Tournament

Code:

```
{% extends "layouts/dashboard_base.html" %}
{% block page_title %}Create Tournament{% endblock %}

{% block content %}
<div class="row align-items-center justify-content-center" style="min-height: 60vh;">
  <div class="col-md-8 text-center">
    <h2 class="fw-bold">Ready to Host?</h2>
    <p class="text-muted mb-4">Configure rules and launch your tournament in minutes.</p>
    <button class="btn btn-primary btn-lg rounded-pill px-5 shadow fw-bold" data-bs-
toggle="modal" data-bs-target="#modalStep1"><i class="fas fa-plus-circle me-2"></i> Create
New Event</button>
  </div>
</div>

<div class="modal fade" id="modalStep1" data-bs-backdrop="static" tabindex="-1">
  <div class="modal-dialog modal-dialog-centered modal-lg">
    <div class="modal-content border-0 shadow-lg rounded-4">
      <div class="modal-header border-0"><h5 class="fw-bold text-primary">Step 1: The
Basics</h5><button type="button" class="btn-close" data-bs-dismiss="modal"></button></div>
      <div class="modal-body p-4">
        <form id="formStep1">
          <div class="row g-3">
            <div class="col-12"><div class="form-floating"><input type="text" class="form-
control" id="inpTitle" required><label>Tournament Name</label></div></div>
            <div class="col-md-6"><div class="form-floating"><select class="form-select"
id="inpSport" required><option value="" disabled selected>Select...</option>{% for s in sports
```



```

%}<option value="{{ s.id }}" data-name="{{ s.name }}">{{ s.name }}</option>{% endfor
%}</select><label>Sport</label></div></div>
    <div class="col-md-6"><div class="form-floating"><select class="form-select"
id="inpVenue">{% for v in venues %}<option value="{{ v.id }}">{{ v.name }}</option>{%
endfor %}</select><label>Venue</label></div></div>
    <div class="col-md-6"><div class="form-floating"><input type="date"
class="form-control" id="inpDate" required><label>Start Date</label></div></div>
    <div class="col-12"><textarea class="form-control" id="inpDesc"
placeholder="Description" style="height: 80px"></textarea></div>
    </div>
  </form>
</div>
  <div class="modal-footer border-0"><button class="btn btn-light rounded-pill" data-bs-
dismiss="modal">Cancel</button><button class="btn btn-primary rounded-pill px-4"
onclick="goToStep2()">Next <i class="fas fa-arrow-right ms-1"></i></button></div>
  </div>
</div>
<div class="modal fade" id="modalStep2" data-bs-backdrop="static" tabindex="-1">
  <div class="modal-dialog modal-dialog-centered modal-lg">
    <div class="modal-content border-0 shadow-lg rounded-4">
      <div class="modal-header border-0"><h5 class="fw-bold text-primary">Step 2:
Configuration</h5><button type="button" class="btn-close" data-bs-
dismiss="modal"></button></div>
      <div class="modal-body p-4">
        <h6 class="fw-bold border-bottom pb-2 mb-3">Participation Limits</h6>
        <div class="row g-3 mb-4">
          <div class="col-md-4"><label class="small fw-bold">Max Teams</label><input
type="number" class="form-control" id="inpMaxTeams" value="8"></div>
          <div class="col-md-4"><label class="small fw-bold">Squad Size</label><input
type="number" class="form-control" id="inpSquadSize" value="15"></div>
          <div class="col-md-4"><label class="small fw-bold">Min Players</label><input
type="number" class="form-control" id="inpMinPlayers" value="11"></div>
        </div>
        <h6 class="fw-bold border-bottom pb-2 mb-3">Game Rules</h6>
        <div id="sportSpecificContainer" class="row g-3"></div>
      </div>
      <div class="modal-footer border-0"><button class="btn btn-light rounded-pill" data-bs-
target="#modalStep1" data-bs-toggle="modal">Back</button><button class="btn btn-primary
rounded-pill px-4" onclick="goToStep3()">Next <i class="fas fa-arrow-right ms-
1"></i></button></div>
    </div>
  </div>
</div>

```

```

<div class="modal fade" id="modalStep3" data-bs-backdrop="static" tabindex="-1">
  <div class="modal-dialog modal-dialog-centered modal-xl">
    <div class="modal-content border-0 shadow-lg rounded-4">
      <div class="modal-header border-0"><h5 class="fw-bold text-primary">Step 3: Custom
Rules</h5><button type="button" class="btn-close" data-bs-dismiss="modal"></button></div>
      <div class="modal-body p-4 bg-light">
        <div id="customRulesList" class="vstack gap-2"></div>
        <div class="text-center mt-3"><button class="btn btn-outline-primary rounded-pill px-
4" onclick="addCustomRuleRow()"><i class="fas fa-plus me-1"></i> Add Rule</button></div>
        </div>
        <div class="modal-footer border-0">
          <button class="btn btn-light rounded-pill" data-bs-target="#modalStep2" data-bs-
toggle="modal">Back</button>
          <button class="btn btn-success rounded-pill px-5 shadow fw-bold"
onclick="submitTournament()">Launch <i class="fas fa-rocket ms-1"></i></button>
        </div>
      </div>
    </div>
  </div>
</div>
{% endblock %}

{% block scripts %}
<script>
  var formData = {};

  function goToStep2() {
    let title = $('#inpTitle').val(), sport = $('#inpSport').val(), date = $('#inpDate').val();
    if(!title || !sport || !date) return alert('Please fill Title, Sport and Date.');

    formData = { title, sport_id: sport, venue_id: $('#inpVenue').val(), start_date: date,
description: $('#inpDesc').val() };
    loadSportInputs($('#inpSport option:selected').data('name'));
    $('#modalStep1').modal('hide'); $('#modalStep2').modal('show');
  }

  function goToStep3() {
    formData.team_config = { max_teams: $('#inpMaxTeams').val(), squad_size:
$('#inpSquadSize').val(), min_players: $('#inpMinPlayers').val() };
    formData.standard_rules = {};
    $('#sportSpecificContainer input, #sportSpecificContainer select').each(function() {
formData.standard_rules[$(this).prev('label').text()] = $(this).val(); });
    $('#modalStep2').modal('hide'); $('#modalStep3').modal('show');
  }

  function loadSportInputs(name) {
    let c = $('#sportSpecificContainer').empty();

```

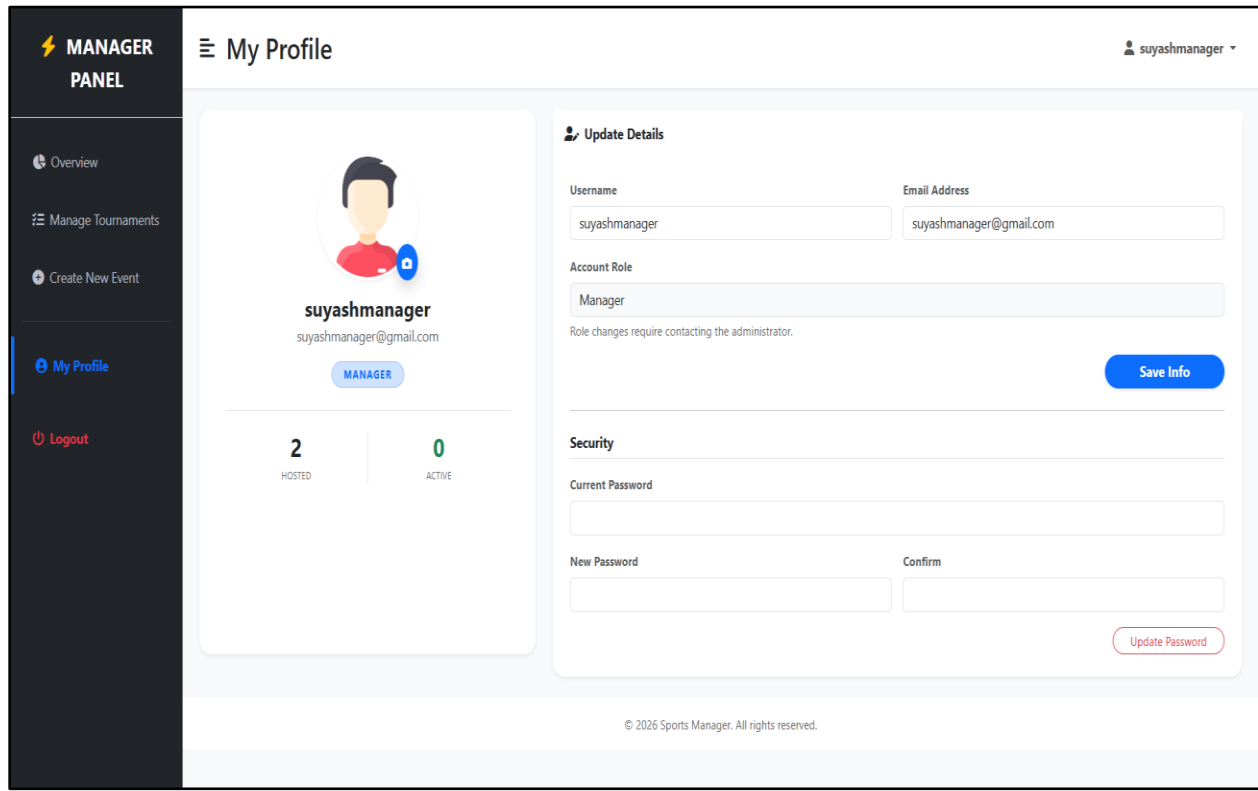
```

const inputs = (l, v) => `

67


```

Manager Profile Tab:



Code:

```
{% extends "layouts/dashboard_base.html" %}
{% block page_title %}My Profile{% endblock %}
{% block content %}
<div class="row justify-content-center">
  <div class="col-lg-4 mb-4">
    <div class="card border-0 shadow-sm rounded-4 text-center h-100">
      <div class="card-body p-5">
        <div class="position-relative d-inline-block mb-3">
          
          <label for="avatarUpload" class="position-absolute bottom-0 end-0 bg-primary text-
white rounded-circle p-2 shadow" style="cursor: pointer;"><i class="fas fa-camera
small"></i></label></div><h4 class="fw-bold mb-1">{{ current_user.username }}</h4>
          <p class="text-muted mb-3">{{ current_user.email }}</p><span class="badge bg-
primary-subtle text-primary border border-primary-subtle rounded-pill px-3 py-2">{{
current_user.role }}</span>

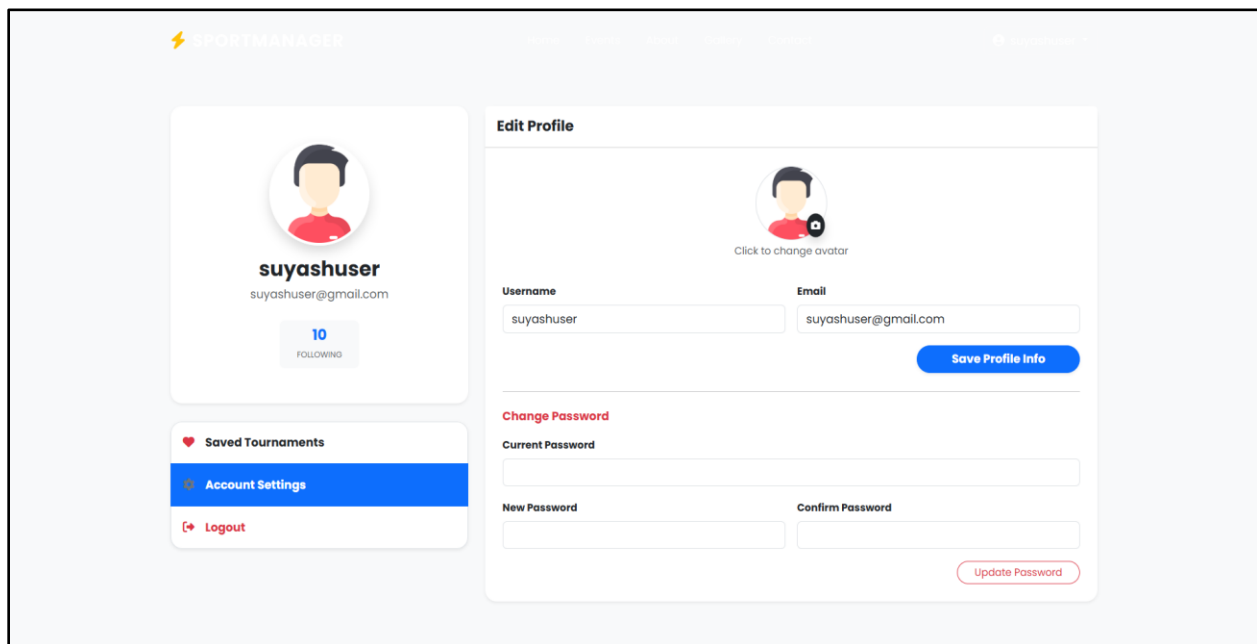
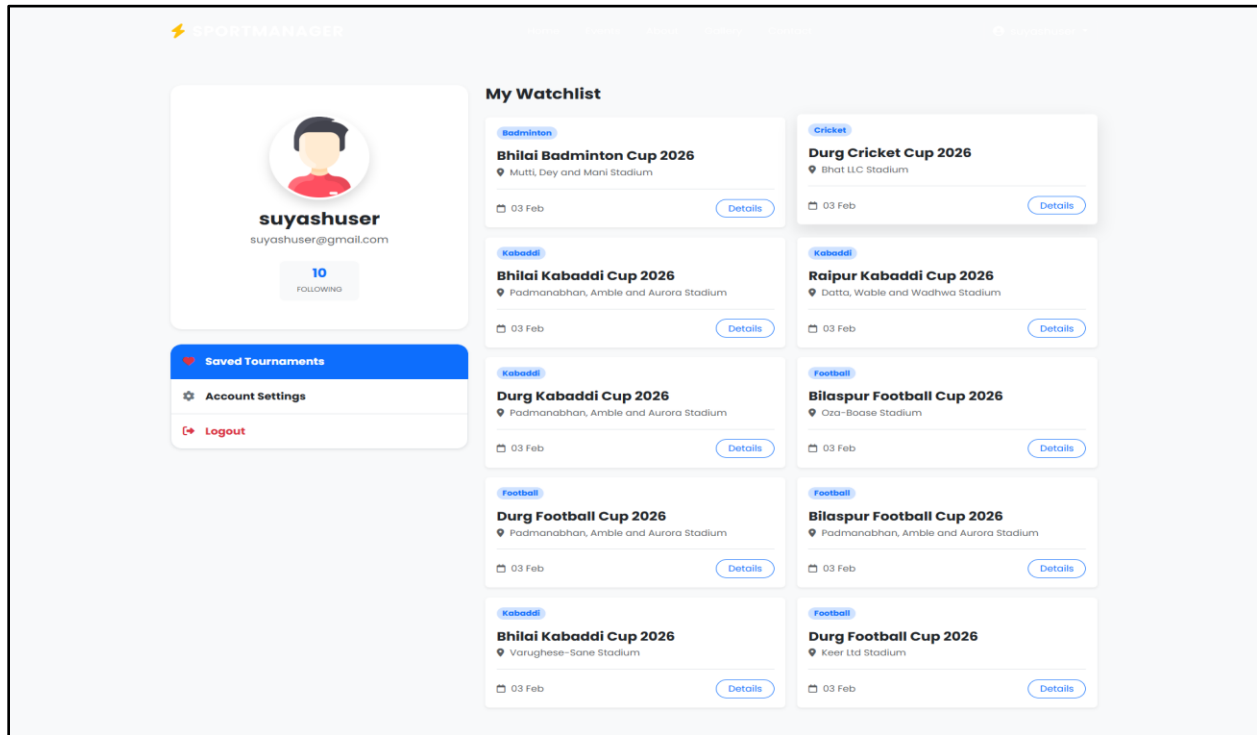
```

```

        <div class="row mt-4 pt-4 border-top">
            <div class="col-6 border-end"><h3 class="fw-bold mb-0 text-dark">{{
stats.total_hosted }}</h3><small class="text-muted">HOSTED</small></div>
            <div class="col-6"><h3 class="fw-bold mb-0 text-success">{{ stats.active_live
}}</h3><small class="text-muted">ACTIVE</small></div>
        </div>
    </div>
</div>
<div class="col-lg-8">
    <div class="card border-0 shadow-sm rounded-4 h-100">
        <div class="card-header bg-white border-bottom-0 py-3"><h6 class="fw-bold mb-0"><i
class="fas fa-user-edit me-2"></i>Update Details</h6></div>
        <div class="card-body p-4">
            <form action="{{ url_for('users.update_profile') }}" method="POST"
enctype="multipart/form-data">
                <input type="file" name="avatar" id="avatarUpload" class="d-none"
onchange="this.form.submit()">
                <div class="row g-3">
                    <div class="col-md-6"><label class="small fw-bold text-
muted">Username</label><input type="text" name="username" class="form-control" value="{{
current_user.username }}" required></div>
                    <div class="col-md-6"><label class="small fw-bold text-
muted">Email</label><input type="email" name="email" class="form-control" value="{{
current_user.email }}" required></div>
                    <div class="col-12 text-end mt-3"><button type="submit" class="btn btn-primary
rounded-pill px-5 fw-bold shadow-sm">Save Info</button></div>
                </div></form><hr class="my-4">
                <form action="{{ url_for('users.change_password') }}" method="POST">
                    <h6 class="fw-bold border-bottom pb-2 mb-3">Security</h6>
                    <div class="row g-3"><div class="col-12"><label class="small fw-bold text-
muted">Current Password</label><input type="password" name="current_password"
class="form-control" required></div>
                    <div class="col-md-6"><label class="small fw-bold text-muted">New
Password</label><input type="password" name="new_password" class="form-control"
required></div><div class="col-md-6"><label class="small fw-bold text-
muted">Confirm</label><input type="password" name="confirm_password" class="form-
control" required></div>
                    <div class="col-12 text-end mt-3"><button type="submit" class="btn btn-outline-
danger btn-sm rounded-pill px-4">Update Password</button></div>
                </div></form>
            </div>
        </div>
    </div>
</div>
{% endblock %}

```

User Profile Page:



Code:

```

{% extends "base.html" %}

{% block content %}
<div style="height: 80px;"></div>

<div class="container py-5">
  <div class="row g-4">
    <div class="col-lg-4">
      <div class="card border-0 shadow-sm rounded-4 overflow-hidden mb-4">
        <div class="card-body text-center p-5">
          <div class="position-relative d-inline-block mb-3">
            
          </div>
          <h3 class="fw-bold mb-1">{{ current_user.username }}</h3>
          <p class="text-muted mb-4">{{ current_user.email }}</p>
          <div class="d-flex justify-content-center gap-2">
            <div class="bg-light rounded-3 px-4 py-2">
              <h5 class="fw-bold mb-0 text-primary">{{ current_user.saved_events|length
            }}</h5>
              <small class="text-muted text-uppercase" style="font-size:
0.7rem;">Following</small>
            </div>
          </div>
          <div class="list-group shadow-sm rounded-4 overflow-hidden border-0">
            <a href="#tab-saved" class="list-group-item list-group-item-action p-3 active fw-bold"
data-bs-toggle="list">
              <i class="fas fa-heart text-danger me-2" style="width: 20px;"></i> Saved
Tournaments
            </a>
            <a href="#tab-settings" class="list-group-item list-group-item-action p-3 fw-bold" data-
bs-toggle="list">
              <i class="fas fa-cog text-secondary me-2" style="width: 20px;"></i> Account
Settings
            </a>
            <a href="{{ url_for('auth.logout') }}" class="list-group-item list-group-item-action p-3
text-danger fw-bold">
              <i class="fas fa-sign-out-alt me-2" style="width: 20px;"></i> Logout
            </a>
          </div>
        </div>
      </div>
    </div>
    <div class="col-lg-8">
      <div class="tab-content">

```

```

<div class="tab-pane fade show active" id="tab-saved">
  <h4 class="fw-bold mb-4">My Watchlist</h4>
  <div class="row g-3">
    {% for event in current_user.saved_events %}
    <div class="col-md-6">
      <div class="card border-0 shadow-sm h-100 hover-lift transition-all">
        <div class="card-body">
          <div class="d-flex justify-content-between mb-3">
            <span class="badge bg-primary-subtle text-primary rounded-pill">{{
event.sport.name }}</span>
            {% if event.status == 'live' %}
            <span class="badge bg-danger">LIVE</span>
            {% endif %}
          </div>
          <h5 class="card-title fw-bold text-dark mb-1">{{ event.title }}</h5>
          <p class="text-muted small mb-3">
            <i class="fas fa-map-marker-alt me-1"></i> {{ event.venue.name if
event.venue else "Venue TBD" }}
          </p>
          <div class="d-flex align-items-center justify-content-between border-top
pt-3 mt-2">
            <div class="text-muted small">
              <i class="far fa-calendar me-1"></i> {{ event.start_date.strftime('%d
%b') }}
            </div>
            <button class="btn btn-sm btn-outline-primary rounded-pill px-3 btn-
details"
              data-event-id="{{ event.id }}">
              Details
            </button>
          </div>
        </div>
      </div>
    {% else %}
    <div class="col-12 text-center py-5 bg-white rounded-4 shadow-sm">
      <div class="mb-3">
        <i class="far fa-heart fa-3x text-light bg-secondary p-4 rounded-circle bg-
opacity-10"></i>
      </div>
      <h5 class="fw-bold">No saved tournaments yet</h5>
      <p class="text-muted mb-4">Follow events to get quick access to scores and
updates.</p>
      <a href="{{ url_for('public.index') }}#events" class="btn btn-primary rounded-
pill px-4">
        Browse Events

```



```

        </a>
    </div>
    {% endfor %}
</div>
</div>
<div class="tab-pane fade" id="tab-settings">
<div class="card border-0 shadow-sm rounded-4">
    <div class="card-header bg-white py-3">
        <h5 class="fw-bold mb-0">Edit Profile</h5>
    </div>
    <div class="card-body p-4">
        <form action="{ { url_for('users.update_profile') } }" method="POST"
enctype="multipart/form-data">
            <div class="row g-3">
                <div class="col-12 text-center mb-3">
                    <label class="d-inline-block position-relative cursor-pointer">
                        
                        <div class="position-absolute bottom-0 end-0 bg-dark text-white rounded-circle
p-2 shadow" style="transform: scale(0.8);">
                            <i class="fas fa-camera"></i>
                        </div>
                        <input type="file" name="avatar" class="d-none"
onchange="document.getElementById('settingsAvatarPreview').src =
window.URL.createObjectURL(this.files[0])">
                    </label>
                    <div class="form-text small">Click to change avatar</div>
                </div>
                <div class="col-md-6">
                    <label class="form-label fw-bold small">Username</label>
                    <input type="text" name="username" class="form-control" value="{ {
current_user.username } }" required>
                </div>
                <div class="col-md-6">
                    <label class="form-label fw-bold small">Email</label>
                    <input type="email" name="email" class="form-control" value="{ {
current_user.email } }" required>
                </div>
                <div class="col-12 text-end mt-3">
                    <button type="submit" class="btn btn-primary rounded-pill px-5 fw-bold shadow-
sm">
                        Save Profile Info
                    </button>
                </div>
            </div>
        </div>
    </div>

```

```

    </form>
    <hr class="my-4 opacity-10">
    <form action="{ { url_for('users.change_password') } }" method="POST">
      <h6 class="fw-bold text-danger mb-3">Change Password</h6>
      <div class="row g-3">
        <div class="col-12">
          <label class="form-label fw-bold small">Current Password</label>
          <input type="password" name="current_password" class="form-control"
required>
        </div>
        <div class="col-md-6">
          <label class="form-label fw-bold small">New Password</label>
          <input type="password" name="new_password" class="form-control"
minlength="6" required>
        </div>
        <div class="col-md-6">
          <label class="form-label fw-bold small">Confirm Password</label>
          <input type="password" name="confirm_password" class="form-control"
minlength="6" required>
        </div>
        <div class="col-12 text-end mt-3">
          <button type="submit" class="btn btn-outline-danger btn-sm rounded-pill px-4">
            Update Password
          </button>
        </div>
      </div>
    </form>
  </div>
</div>
</div>
</div>
</div>
</div>
</div>

{% include 'partials/global_event_modal.html' %}

{% endblock %}

{% block scripts %}
<script>
  // Since we are using the global modal logic defined in app.js,
  // we don't need extra JS here for the modal trigger.
  // The class .btn-details logic handles it.
</script>
{% endblock %}

```

12. LIMITATIONS

1. Dependence on Manual Data Entry: The current system relies entirely on tournament managers to manually input scores for every event. Whether it is recording a "4 runs" in Cricket or a "100kg" attempt in Weightlifting, the data must be typed in real-time. This process is prone to human error and can be tedious during high-paced matches. Unlike professional broadcast systems, there is no integration with automated sensors or smart-stadium APIs to capture data automatically.

2. No Offline Functionality: The platform operates exclusively as a web application requiring an active internet connection. If a tournament is being held in a remote venue (e.g., a rural stadium) with poor connectivity, managers cannot log scores locally and sync them later. This limits the system's reliability in areas with unstable network infrastructure.

3. Lack of Live Video Streaming: Currently, the system provides only text-based updates (Live Scorecards). It does not support real-time video streaming or integration with platforms like YouTube Live or Twitch. Spectators can see the score change, but they cannot watch the actual gameplay within the dashboard.

4. Limited Advanced Analytics: While the system tracks basic metrics (Runs, Wickets, Goals, Lifts), it lacks deep statistical analysis. Features like "Player Heatmaps" for football, "Wagon Wheels" for cricket, or "Predictive Match Outcomes" based on historical performance are not currently implemented.

13. FUTURE SCOPE OF THE PROJECT

1. Integration with IoT & Smart Sensors: Future versions of the system could integrate with Internet of Things (IoT) devices. For example, connecting to "Smart Stumps" in cricket to detect wickets automatically, or digital weight scales in weightlifting to auto-populate the lifted weight. This would significantly reduce manual effort and increase data precision.

2. Live Streaming Integration: The platform can be expanded to support embedded live streams. By integrating with broadcasting APIs (like OBS Studio or YouTube Data API), organizers could link a live video feed directly to the match page, allowing spectators to watch the game alongside the real-time scoreboard.

3. Native Mobile Application: Developing a dedicated mobile app (iOS/Android) using frameworks like Flutter or React Native would provide a better experience for field managers. Key features would include **Offline Mode**, allowing scores to be entered without internet and synced once connectivity is restored, and **Push Notifications** to alert fans about match start times or wicket falls.

4. Payment Gateway for Registrations: Currently, team registration is administrative. The system could integrate payment gateways (like Razorpay or Stripe) to allow teams to pay participation fees online during the registration process, making the platform a complete revenue management tool for organizers.

5. AI-Driven Performance Analysis: By implementing Machine Learning algorithms, the system could analyze player data over multiple seasons to provide "Scouting Reports." This would help

coaches identify top-performing athletes based on calculated metrics (e.g., "Strike Rate in Death Overs" or "Consistency in Snatch Attempts").

14. CONCLUSION

The **Sports Management System** project successfully addresses the key objectives of providing a comprehensive and user-friendly platform for modern tournament administration. By offering specialized tools for managing diverse sports like Cricket, Football, and Weightlifting within a single unified interface, the platform empowers organizers to move away from chaotic paper-based methods to a structured digital workflow.

The scope of the project included creating a robust system that manages the complex hierarchy of Sports, Events, Teams, and Players. The technical architecture, built on **Python (Flask)** and **MySQL**, utilizes a unique **JSON-based schema approach** to handle the varying rules of different sports without rigid database constraints. The **Role-Based Access Control (RBAC)** ensures that data integrity is maintained, giving Managers full control over their events while providing Public Users with instant, read-only access to live results.

In conclusion, this project provides an efficient, scalable, and secure solution for digitizing sports events. It meets the platform's goals by ensuring accurate score calculation, conflict-free scheduling, and instant visual feedback for fans. With the potential for future enhancements like mobile apps and automated analytics, this system is poised to become a valuable asset for sports associations and college tournament directors.

15. BIBLIOGRAPHY

The Bibliography contains references to all the documents, technical documentation, and learning resources that were utilized for the creation and successful completion of the **Sports Management System**.

- **Book:** *Flask Web Development: Developing Web Applications with Python* by Miguel Grinberg.
- **Book:** *Python Crash Course* by Eric Matthes (Section on Web Apps and Database Management).
- **Documentation:** Flask Official Documentation (<https://flask.palletsprojects.com/>).
- **Documentation:** SQLAlchemy ORM Documentation (<https://www.sqlalchemy.org/>) – Specifically for Many-to-Many relationships and JSON column types.
- **Documentation:** Jinja2 Template Designer Documentation (For dynamic HTML rendering).
- **Resource:** <https://getbootstrap.com> (Bootstrap 5 Documentation for Responsive Dashboard Design).
- **Resource:** <https://jquery.com> (jQuery Documentation for AJAX and DOM Manipulation).
- **Tool:** <https://fontawesome.com> (Icons for Sports and Actions).
- **Tool:** <https://mermaid.live> (Used for generating DFD and ER Diagrams).
- **Tutorials:** "Corey Schafer - Python Flask Series" on YouTube (For Authentication and Blueprint structure).